

Anexo II
Dpto. Informática – CIFP César Manrique
Módulo Proyecto
CFGS Desarrollo de Aplicaciones Web

TÁBULA

SISTEMA DE GESTIÓN DE INVENTARIOS CON
DASHBOARD EN TIEMPO REAL

Autor/es: Nazaret Duque Díaz
Profesor: José David Díaz Díaz
Fecha: 05/05/2025

ÍNDICE

INTRODUCCIÓN.....	4
ORIGEN Y CONTEXTUALIZACIÓN DEL PROYECTO.....	5
¿Por qué utilizar Tábula?.....	5
OBJETIVO GENERAL DEL PROYECTO.....	6
OBJETIVOS ESPECÍFICOS.....	6
TAREAS.....	7
TAREA 1: Análisis inicial y diseño de la arquitectura del sistema.....	7
Subtarea 1.1: Investigación sobre tecnologías y frameworks.....	7
Subtarea 1.2: Definición de requisitos funcionales y no funcionales.....	9
Requisitos funcionales.....	9
Requisitos no funcionales.....	10
Subtarea 1.3: Diseño inicial de la arquitectura del sistema.....	11
Duración.....	13
TAREA 2: Desarrollo del Backend (Base de Datos y API REST).....	14
Subtarea 2.1: Configuración del entorno de desarrollo.....	14
Subtarea 2.2: Diseño e implementación de la base de datos (PostgreSQL).....	15
Subtarea 2.2.1 Configuración inicial.....	15
Subtarea 2.2.2 Modelo relacional y entidades.....	15
Subtarea 2.2.3 Trigger automático de stock.....	18
Subtarea 2.3: Estructuración del backend y preparación del entorno de desarrollo.....	20
Subtarea 2.3.1 Preparación del entorno virtual.....	21
Subtarea 2.3.2 Instalación de dependencias.....	22
Subtarea 2.4: Desarrollo de la API REST con FastAPI.....	23
Subtarea 2.4.1: Modelos, esquemas y conexión con la base de datos.....	23
Subtarea 2.4.2: Implementación de autenticación con JWT.....	31
Subtarea 2.4.3: Diseño del resto de endpoints y CRUDs.....	39
Duración.....	50
Tarea 3. Desarrollo del Frontend (React y Tailwind).....	51
Subtarea 3.1: Formación y configuración del entorno frontend.....	51
Subtarea 3.1.1 Formación.....	51
Subtarea 3.1.2: Configuración del entorno de desarrollo.....	52
Subtarea 3.2: Diseño general de la interfaz de usuario (UI).....	56
Subtarea 3.2.1: Guía de estilos y personalización visual con Tailwind CSS.....	57
Subtarea 3.2.2: Usabilidad y accesibilidad.....	59
Subtarea 3.2.3: Vistas tipo y componentes principales.....	60
Subtarea 3.3: Implementación de la lógica de autenticación y rutas.....	74
Subtarea 3.3.1: Estructura base y montaje de la aplicación.....	74
Subtarea 3.3.2: Contexto de autenticación (src/context/AuthProvider).....	75
Subtarea 3.3.3: Sistema de rutas protegidas y navegación condicional.....	76
Subtarea 3.4: Implementación de vistas dinámicas y componentes funcionales con React.....	79
Subtarea 3.4.1: Vistas <i>de la aplicación</i> , lógica reactiva y uso de hooks.....	79
Subtarea 3.4.2: Componentes de la aplicación.....	81

Duración.....	86
Tarea 4: WebSocket y Despliegue.....	87
Subtarea 4.1: Desarrollo de la funcionalidad de WebSocket.....	87
Subtarea 4.1.1: Backend.....	87
Subtarea 4.1.2: Frontend.....	89
Subtarea 4.2: Refactorización del código y testeo.....	91
Subtarea 4.3: Creación de repositorios públicos en GitHub.....	92
Subtarea 4.4: Configuración del hosting en Render y Vercel.....	93
Subtarea 4.4.1: Backend – Despliegue en Render.....	93
Subtarea 4.4.2: Frontend – Despliegue en Vercel.....	95
Duración.....	96
RECURSOS HUMANOS.....	97
RECURSOS MATERIALES.....	98
CRONOGRAMA.....	99
PRESUPUESTO.....	100
POLÍTICA DE SEGUIMIENTO Y EVALUACIÓN.....	102
Futuras mejoras.....	102
ANEXOS.....	104
BIBLIOGRAFÍA.....	105
Recursos y referencias textuales externas.....	105
Documentación oficial y guías de las tecnologías y herramientas del proyecto.....	106
Imágenes y recursos visuales (licencias y atribuciones):.....	106

INTRODUCCIÓN

El presente documento recoge el desarrollo del proyecto de creación de **Tábula**, una aplicación web para la gestión de inventario de pequeñas y medianas empresas.

Tábula toma su nombre de las *tabulae*^[1] romanas, las primeras herramientas de registro manual utilizadas por escribas para administrar los recursos del Estado, los templos y el comercio en la Antigua Roma. Inspirándose en esta tradición histórica, el nombre elegido busca rendir homenaje a los orígenes de la contabilidad y la logística.

El objetivo de **Tábula** es ofrecer una solución sencilla, accesible y efectiva para el control de inventarios, incorporando funcionalidades clave como la gestión de movimientos de stock, actualizaciones en tiempo real mediante WebSockets y la visualización de métricas a través de dashboards interactivos.

Este documento expone de manera estructurada las tareas realizadas durante el desarrollo de la aplicación, los recursos utilizados, la planificación seguida, el presupuesto estimado y las acciones propuestas para el seguimiento y evaluación del proyecto.

Se incluyen referencias a materiales anexados creados durante el proceso, tales como scripts de base de datos, código fuente del backend y frontend, documentación técnica, etc.

ORIGEN Y CONTEXTUALIZACIÓN DEL PROYECTO

La necesidad de optimizar la gestión de inventario en pequeñas y medianas empresas fue el principal motor de este proyecto. En muchos casos, los métodos tradicionales de control (en muchas ocasiones simples hojas de cálculo) generan errores, pérdidas de información o falta de visibilidad sobre el estado del stock.

Aunque existen soluciones comerciales en el mercado, muchas de ellas presentan costes elevados o niveles de complejidad innecesarios para pequeñas organizaciones. De ahí surge la idea de desarrollar una herramienta adaptada a este tipo de necesidades específicas, utilizando tecnologías modernas y de código abierto.

¿Por qué utilizar Tábula?

Tábula proporciona una forma sencilla y visual de gestionar el inventario de una empresa en tiempo real.

¿Qué permite hacer?

- Registrar entradas y salidas de stock de forma rápida y organizada.
- Consultar el estado actual del inventario en cualquier momento.
- Visualizar métricas y tendencias de stock mediante dashboards interactivos.
- Mejorar la planificación y el control del inventario diario.

El enfoque de **Tábula** es ofrecer una solución accesible y eficaz, centrada exclusivamente en la gestión de stock, sin añadir funcionalidades adicionales como venta, compra o incidencias de producto, para asegurar su simplicidad y facilidad de uso.

OBJETIVO GENERAL DEL PROYECTO

Desarrollar una aplicación web escalable y segura que permita a pequeñas y medianas empresas gestionar su inventario de manera eficaz, incluyendo funcionalidades de control de stock, movimientos de inventario, notificaciones en tiempo real y visualización de datos mediante gráficos interactivos.

OBJETIVOS ESPECÍFICOS

Entre los objetivos específicos de este proyecto se destacan a continuación los siguientes puntos:

- Permitir el registro, modificación y eliminación de productos en el inventario.
- Gestionar entradas y salidas de stock de forma segura y eficiente.
- Proporcionar actualizaciones en tiempo real mediante el uso de WebSockets.
- Visualizar métricas de inventario y tendencias a través de gráficas en un dashboard interactivo.
- Garantizar la seguridad de acceso mediante autenticación JWT.
- Implementar un sistema modular que facilite el futuro despliegue y la escalabilidad.

TAREAS

TAREA 1: Análisis inicial y diseño de la arquitectura del sistema

Esta tarea consiste en definir la estructura general de la aplicación, seleccionar las tecnologías adecuadas y diseñar la arquitectura del sistema, asegurando una base sólida para el desarrollo posterior.

Metodología:

- Realizar una investigación inicial sobre tecnologías backend, frontend, base de datos y comunicación en tiempo real.
- Definir los requisitos funcionales y no funcionales que debe cumplir la aplicación.
- Diseñar la arquitectura general del sistema, incluyendo la interacción entre sus diferentes componentes (backend, frontend, base de datos, WebSocket).
- Crear esquemas iniciales que representen visualmente la arquitectura propuesta.

La metodología seguida ha sido incremental: partiendo de una planificación general, se fueron afinando los detalles técnicos a medida que avanzaba el desarrollo.

Subtarea 1.1: Investigación sobre tecnologías y frameworks

Antes de iniciar el desarrollo técnico de la aplicación, se llevó a cabo una fase de investigación orientada a identificar las tecnologías más adecuadas para cada parte del sistema. Se buscó un equilibrio entre sencillez de uso, documentación disponible, compatibilidad entre herramientas y posibilidad de escalar o mantener el proyecto a medio plazo. También se tuvo en cuenta la curva de aprendizaje, ya que el objetivo era poder abarcar todo el desarrollo de la aplicación dentro del calendario previsto.

Evaluación de tecnologías:

- **Backend:**

Como lenguaje principal para el backend se eligió **Python** debido a su gran ecosistema de librerías. A partir de esta elección, se investigaron diferentes frameworks y tecnologías para cada parte del sistema, evaluando su idoneidad según criterios de sencillez, curva de aprendizaje, compatibilidad entre herramientas y documentación disponible.

Se compararon opciones como Flask, Django y FastAPI.

- *Flask* ofrece simplicidad, pero requiere muchas decisiones manuales para una app compleja.
- *Django* es un framework completo, pero su estructura rígida y curva de aprendizaje inicial lo hacían menos adecuado para un proyecto de tiempo limitado.
- [*FastAPI*](#) destacó por su facilidad de uso, rendimiento, y sobre todo por su integración nativa con Pydantic y tipado automático, lo que facilitó tanto el desarrollo como la documentación automática de la API.

- **Frontend:**

Se consideraron React y Vue.js.

- [*React*](#) fue la opción elegida por su gran comunidad, abundante documentación y compatibilidad con librerías como [*Recharts*](#) para visualización de datos.
- Su integración con herramientas modernas como [*Vite*](#) y [*Tailwind*](#) permitió un desarrollo ágil y un diseño coherente.

- **Base de datos:**

Se valoraron PostgreSQL y MongoDB.

- *MongoDB*, aunque más flexible para estructuras no relacionales, no se ajustaba bien al modelo de inventario propuesto, que depende de relaciones claras entre productos, almacenes, movimientos, etc.

- [PostgreSQL](#) resultó la mejor opción por su solidez, compatibilidad con ORMs relacionales y soporte para funcionalidades como triggers y restricciones.
- **Tiempo real (WebSockets):**

Se investigó el uso de WebSocket nativo, Socket.IO y SignalR.

 - Se optó por el **WebSocket nativo integrado en FastAPI**, por ser suficiente para los requisitos del proyecto y evitar dependencias externas innecesarias.

Resumen de elección de tecnologías:

Finalmente, se eligió un stack tecnológico moderno, equilibrando entre potencia y sencillez.

- **Backend:** FastAPI + SQLAlchemy + SQLModel
- **Frontend:** React + Tailwind CSS + Recharts
- **Base de datos:** PostgreSQL
- **Tiempo real:** WebSocket nativo
- **Control de versiones:** [Git](#) + [GitHub](#)

Subtarea 1.2: Definición de requisitos funcionales y no funcionales

Se estableció de forma clara lo que la aplicación debía hacer (requisitos funcionales) y cómo debía hacerlo (requisitos no funcionales), definiendo así el alcance inicial del proyecto y sentando las bases para el desarrollo.

Requisitos funcionales

Son las funcionalidades concretas que el sistema debe proporcionar:

1. **Autenticación de usuarios** mediante credenciales (email y contraseña).
2. **Roles de usuario:** rol *admin* con permisos ampliados y rol *usuario* con permisos limitados.

3. Gestión de productos:

- Crear, editar, eliminar y listar productos.
- Asignación de categorías.
- Activar o desactivar productos.

4. Gestión de almacenes:

- Crear, editar, eliminar y listar almacenes.
- Activar o desactivar almacenes.

5. Movimientos de stock:

- Registrar entradas y salidas de productos.
- Asignar productos a almacenes con lote y fecha de caducidad.
- Visualizar historial de movimientos.

6. Cálculo automático del stock actual en función de los movimientos registrados.**7. Dashboard visual** con métricas del inventario:

- Niveles de stock por almacén, producto o categoría.
- Semáforo con productos próximos a caducar.

8. Notificaciones en tiempo real de nuevos movimientos de stock mediante WebSockets.**9. Filtrado y búsqueda** en listados de productos, almacenes y movimientos.**10. Exportación de datos a CSV** para productos y movimientos.***Requisitos no funcionales***

Son criterios que definen la **calidad, rendimiento, seguridad o limitaciones técnicas** del sistema:

- 1. Facilidad de uso:** interfaz sencilla, clara y accesible.
- 2. Seguridad:**

- Contraseñas cifradas.
 - Uso de JWT para autenticación.
 - Protección de endpoints según rol del usuario.
3. **Escalabilidad técnica:** arquitectura separada por capas (frontend, backend, base de datos), preparada para ser desplegada con Docker (aunque esta parte no se llegó a completar durante la fase de desarrollo).
 4. **Compatibilidad** con navegadores modernos y dispositivos de escritorio.
 5. **Código abierto y modular**, alojado en GitHub.
 6. **Desempeño aceptable:** respuesta rápida en navegación y peticiones.
 7. **Actualización en tiempo real** usando WebSocket sin recargas.

Subtarea 1.3: Diseño inicial de la arquitectura del sistema

Tras la definición de requisitos y la selección de tecnologías, se procedió al diseño de la arquitectura general que guiaría el desarrollo del sistema. El objetivo era establecer una estructura clara, modular y escalable, que facilitara tanto la implementación inicial como su posible crecimiento futuro.

La aplicación se diseñó siguiendo una arquitectura **full stack** dividida en tres capas principales: **frontend, backend y base de datos**. Adicionalmente, se contempló la integración de un servidor de WebSockets para habilitar notificaciones en tiempo real del estado del inventario.

El siguiente esquema refleja la interacción entre los distintos componentes de la aplicación.

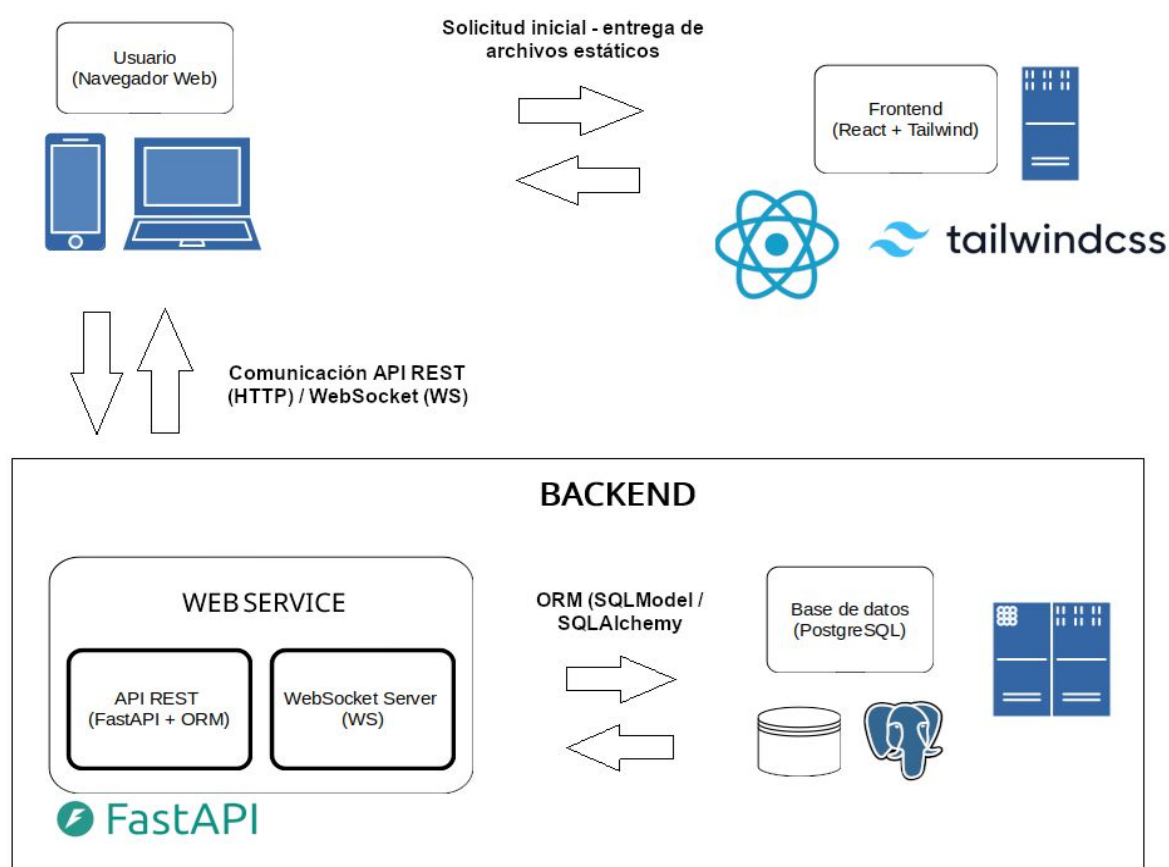


Figura 1: Esquema de arquitectura de aplicación web Tábula, elaboración propia.

Los logotipos de React, Tailwind CSS, FastAPI y PostgreSQL utilizados en la figura han sido extraídos de Wikimedia Commons y fuentes oficiales, integrados bajo licencias compatibles. Las atribuciones completas se detallan en la sección "Imágenes y recursos visuales".

Cuando el usuario accede a la aplicación, el navegador realiza una solicitud inicial para cargar los archivos estáticos (HTML, CSS y JavaScript) que componen la interfaz de usuario. Esta entrega se realiza desde un servidor de archivos estáticos, que puede ser Vite en modo desarrollo o plataformas como Vercel en producción. Una vez cargado el frontend en el navegador, la aplicación interactúa dinámicamente con el backend mediante peticiones HTTP (API REST) y mantiene una conexión activa mediante WebSocket para recibir notificaciones en tiempo real.

Duración

	Duración (horas)	Fecha inicio	Fecha final
Tarea 1. Fase de Planificación		17/02/2025	23/02/2025
Subtarea 1. 1: Investigación sobre tecnologías y frameworks.	2.5	17/02/2025	19/02/2025
Subtarea 1. 2: Definición de requisitos funcionales y no funcionales.	1.5	18/02/2025	20/02/2025
Subtarea 1. 3: Diseño inicial de la arquitectura del sistema	2	20/02/2025	23/02/2025
TOTAL	6		

TAREA 2: Desarrollo del Backend (Base de Datos y API REST)

Subtarea 2.1: Configuración del entorno de desarrollo

Para llevar a cabo el desarrollo del backend se establece un entorno de trabajo compatible con los distintos sistemas operativos utilizados a lo largo del proyecto: **Linux Mint** y **Windows 11**. En ambos entornos se trabaja con [Visual Studio Code](#) como editor principal, junto con extensiones y herramientas de apoyo como [Thunder Client](#), empleada para realizar pruebas de peticiones a la API REST.

La base de datos se gestiona en local mediante **PostgreSQL**, accediendo a ella a través de [PgAdmin 4](#), una interfaz gráfica que facilita tanto el diseño como la consulta y edición de datos. La creación de tablas, relaciones y el trigger de stock se lleva a cabo desde esta herramienta.

Se utiliza **Git** como sistema de control de versiones, en combinación con **GitHub** para alojar el proyecto en un repositorio privado. En el entorno Windows, se emplea **Git Bash** para trabajar con comandos de consola.

Para el desarrollo en Python se crean **entornos virtuales** independientes, lo que permite aislar las dependencias del proyecto y asegurar una mayor estabilidad durante la instalación y ejecución de paquetes.

Este entorno técnico permite trabajar de forma organizada, realizar pruebas rápidas y mantener un control eficaz sobre la evolución del código y la estructura del sistema.

Subtarea 2.2: Diseño e implementación de la base de datos (PostgreSQL)

Subtarea 2.2.1 Configuración inicial

La base de datos del sistema se gestiona mediante **PostgreSQL** (versión 17.4-1), instalada localmente y administrada a través de la interfaz gráfica **PgAdmin 4**. Para el desarrollo, se creó un entorno básico compuesto por:

- **Base de datos:** inventario
- **Usuario administrador:** admin
- **Contraseña:** admin123

Aunque las tablas estaban diseñadas para generarse automáticamente mediante ORM desde el backend (con `SQLModel`), se optó inicialmente por definir las manualmente en SQL para validar el modelo relacional de forma directa y comenzar el trabajo práctico de forma inmediata. Esta estructura inicial permitió ejecutar consultas, diseñar relaciones y probar el funcionamiento del sistema antes de integrar el backend.

Subtarea 2.2.2 Modelo relacional y entidades

El sistema de gestión de inventario se apoya en un modelo relacional compuesto por varias entidades principales. Cada una cumple un rol específico dentro de la lógica del sistema:

- **Usuario (usuario):** representa a los usuarios que interactúan con el sistema. Cada usuario tiene un rol (por ejemplo, administrador o usuario normal) y puede ser responsable de registrar movimientos de stock.
- **Producto (producto):** almacena la información de los productos gestionados, incluyendo un código identificador, un SKU único, una descripción y una referencia a la categoría a la que pertenecen. También se contempla un estado de activación (activo) para permitir su desactivación sin eliminarlo.

- **Categoría de producto (categoria_producto):** permite clasificar los productos en diferentes grupos (alimentación, higiene, etc.). Está vinculada a los productos mediante una relación 1:N.
- **Almacén (almacen):** representa los distintos lugares físicos donde se almacenan los productos. Cada almacén tiene una descripción y puede estar activo o inactivo.
- **Movimiento (movimientos):** registra una acción de entrada o salida de stock, asociada a un usuario y a una fecha. Sirve como entidad principal para organizar el historial de operaciones.
- **Línea de movimiento (movimientos_lineas):** detalla los productos, cantidades, lotes y almacenes implicados en cada movimiento. Cada línea está vinculada a un movimiento y a una combinación específica de producto, almacén y lote.
- **Stock (stock):** refleja el estado actual del inventario, agrupado por producto, almacén y lote. Esta tabla no se edita directamente, sino que se actualiza automáticamente mediante un trigger asociado a los movimientos.

A continuación se muestra un resumen de las tablas y sus relaciones clave:

Tabla	Atributos	Relaciones
usuario	id (PK), nombre, email, passwd, rol, activo	- Relación uno a muchos (1:N) con movimientos (un usuario puede realizar múltiples movimientos).
almacen	codigo (PK), descripcion, activo	- Relación uno a muchos (1:N) con movimientos_lineas (un almacén puede estar en muchos movimientos).
categoria_producto	Id (PK), nombre	- Relación uno a muchos (1:N) con productos (una misma categoría contiene muchos productos)
producto	codigo (PK), sku, nombre_corto, descripcion, id_categoria (FK), activo	- Relación uno a muchos (1:N) con movimientos_lineas (un producto puede aparecer en múltiples movimientos). - Relación uno a uno (1:1) con categoria_producto (un producto solo puede pertenecer a una categoría)
movimientos	id_mov (PK), fecha, tipo (entrada/salida), id_usuario (FK)	- Relación muchos a uno (N:1) con usuario (cada movimiento es realizado por un usuario). - Relación uno a muchos (1:N) con movimientos_lineas (un movimiento puede tener varias líneas)
movimientos_lineas	id_linea (PK), id_mov (PK, FK), codigo_almacen (FK), codigo_producto (FK), lote, fecha_cad, cantidad	- Relación muchos a uno (N:1) con movimientos (una línea pertenece a un solo movimiento). - Relación muchos a uno (N:1) con producto y almacen.
stock	codigo_almacen (PK, FK), codigo_producto (PK, FK), lote (PK), fecha_cad, cantidad	- Relación muchos a uno (N:1) con producto y almacen (un producto puede estar en varios almacenes con diferentes lotes).

Este modelo queda representado gráficamente en el **esquema ERD** siguiente:



Figura 2: Esquema ERD de la base de datos, generado mediante PgAdmin

Subtarea 2.2.3 Trigger automático de stock

Para mantener la coherencia entre los movimientos registrados y el stock disponible en el sistema, se implementa un **trigger** en la base de datos que se activa automáticamente tras cada inserción en la tabla `movimientos_lineas`. Este trigger ejecuta la función `actualizar_stock()`, definida en SQL, que se encarga de modificar la tabla `stock` en función del tipo de movimiento.

El funcionamiento del trigger es el siguiente:

- **Identificación del tipo de movimiento:** La función consulta en la tabla `movimientos` si el movimiento insertado es de tipo entrada o salida, y extrae además el identificador del usuario que lo realiza.
- **Normalización del lote:** Si no se especifica lote, se asigna el valor 'SIN_LOTE'. Además, se validan dos reglas de integridad:

- No se permite registrar una fecha de caducidad si el lote es 'SIN_LOTE'.
- No se permite que un mismo lote tenga distintas fechas de caducidad en la base de datos.
- **Entradas de stock:** Si el movimiento es de tipo entrada, se intenta insertar una nueva fila en stock. En caso de conflicto (es decir, si ya existe una fila con la misma combinación de almacén, producto y lote), se actualiza la cantidad sumando la nueva cantidad registrada.
- **Salidas de stock:** Si el movimiento es de tipo salida, se verifica primero que haya suficiente stock disponible. Si no existe una fila correspondiente, o si la cantidad es insuficiente, se lanza una excepción. En caso contrario, se actualiza la fila restando la cantidad, sin permitir que el valor final baje de cero (GREATEST(0, ...)).

Esta lógica garantiza que:

- El sistema nunca permita salidas de stock no válidas.
- No se pueda registrar información contradictoria sobre fechas de caducidad.
- El stock refleje siempre la realidad tras cada movimiento.

El código fuente del trigger se encuentra en el archivo anexo junto con el resto de ficheros del proyecto: [tabula-backend/db/db_trigger.sql](#)

No se incluirán los contenidos completos del código fuente del proyecto en el documento para no sobrecargarlo, pero dicho código fuente se documenta internamente mediante comentarios y puede consultarse como material complementario, ver apartado de [ANEXOS](#).

Subtarea 2.3: Estructuración del backend y preparación del entorno de desarrollo

Antes de comenzar con la implementación de la API REST, se definió la estructura de carpetas y archivos del backend siguiendo las [recomendaciones oficiales de FastAPI^{\[2\]}](#) y [otros recursos^{\[3\]}](#) de buenas prácticas. El objetivo era organizar el código de forma modular y mantenible, facilitando el desarrollo progresivo de modelos, rutas, lógica de autenticación y dependencias.

Aunque no se sigue estrictamente un patrón *Modelo-Vista-Controlador* (MVC), la organización adoptada se inspira en dicha separación de responsabilidades. En este caso:

- Los modelos de datos (entidades de base de datos) se encuentran en la carpeta `models/`.
- Los esquemas de validación de entrada y salida (usando Pydantic) están en `schemas/`.
- Las rutas (endpoints) que actúan como controladores lógicos del comportamiento están en `routers/`.
- La carpeta `utils/` contiene funciones auxiliares y utilidades diversas.
- Las dependencias reutilizables (como comprobaciones de rol) se ubican en `dependencies.py`.
- El archivo `main.py` actúa como punto de entrada de la aplicación, donde se inicializa FastAPI y se cargan los routers, middleware y configuraciones generales.

La estructura final del backend quedó organizada de la siguiente manera:

```
tabula-backend/  
├── app/  
│   ├── dependencies.py  
│   ├── main.py  
│   ├── models/  
│   ├── routers/  
│   ├── schemas/  
│   └── utils/  
├── db/  
│   ├── db_tables_trigger.sql  
│   └── db_trigger.sql  
├── LICENSE  
├── README.md  
└── requirements.txt
```

Subtarea 2.3.1 Preparación del entorno virtual

Siguiendo las [recomendaciones de FastAPI^{\[4\]}](#), se trabajó dentro de un entorno virtual aislado para gestionar correctamente las dependencias del proyecto y evitar conflictos con otras instalaciones de Python. Este entorno se creó utilizando el módulo venv.

En Windows, desde Git Bash:

```
py -m venv .venv  
source .venv/Scripts/activate
```

En Linux:

```
python3 -m venv .venv  
source .venv/bin/activate
```

Para salir del entorno virtual, se utiliza el comando **deactivate**.

Subtarea 2.3.2 Instalación de dependencias

Una vez activado el entorno virtual, se instalaron los paquetes necesarios mediante pip. Las dependencias principales utilizadas fueron:

- `fastapi[standard]`: framework para construir APIs rápidas y modernas.
- `sqlmodel`: ORM que combina SQLAlchemy y Pydantic para trabajar con bases de datos SQL.
- `pyjwt`: manejo de tokens JWT para autenticación.
- `passlib[bcrypt]`: cifrado seguro de contraseñas con el algoritmo bcrypt.
- `psycopg2-binary`: driver de PostgreSQL para conectar con la base de datos desde Python.
- `python-dotenv`: permite cargar variables de entorno desde un archivo `.env`.

Los comandos utilizados fueron:

```
py -m pip install "fastapi[standard]"
```

```
py -m pip install sqlmodel pyjwt "passlib[bcrypt]" psycopg2-binary python-dotenv
```

Una vez instaladas las dependencias, se generó el archivo [`requirements.txt`](#)^[5] con el comando:

```
pip freeze > requirements.txt
```

Esto permite que cualquier otro entorno pueda reproducirse fácilmente con:

```
pip install -r requirements.txt
```

Subtarea 2.4: Desarrollo de la API REST con FastAPI

El objetivo de esta tarea fue construir una API REST que sirviera como puente entre la base de datos y el frontend, permitiendo a los usuarios realizar operaciones como gestionar productos, almacenes, movimientos de stock, consultar métricas, etc.

El desarrollo de esta API se dividió en varias subtareas, que abarcan desde la definición de modelos de datos y esquemas de validación hasta la implementación de autenticación, lógica de negocio y definición de los endpoints.

Subtarea 2.4.1: Modelos, esquemas y conexión con la base de datos

Uso de SQLAlchemy y diseño del modelo de datos

Para trabajar con PostgreSQL desde FastAPI se utilizó la librería **SQLModel**, una herramienta moderna que combina lo mejor de **SQLAlchemy** (ORM tradicional) y **Pydantic** (validación de datos). Esta elección fue coherente con las [recomendaciones oficiales de FastAPI](#)^[6], ya que SQLModel permite definir en una sola clase tanto la estructura de las tablas como los modelos usados para manipular los datos dentro del código.

Todos los modelos fueron definidos en la carpeta [tabula-backend/app/models](#). Cada archivo representa una entidad clave del sistema, como User, Product, Movement, Stock, etc.

Ejemplo: modelo para la tabla producto:

```

app > models > product.py > ...
1 from typing import Optional
2 from sqlmodel import SQLModel, Field
3
4
5 class Product(SQLModel, table=True):
6     __tablename__ = "producto"
7
8     codigo: int = Field(default=None, primary_key=True, nullable=False)
9     sku: str = Field(unique=True, index=True, nullable=False)
10    nombre_corto: str = Field(nullable=False)
11    descripcion: Optional[str] = Field(default=None)
12    id_categoria: int = Field(foreign_key="categoria_producto.id", nullable=False)
13    activo: bool = Field(default=True, nullable=False)
14

```

Figura 3: Captura tomada desde VS Code del código para el modelo "Product"

Explicación del código:

- `class Product(SQLModel, table=True)`: Al heredar de `SQLModel` con el parámetro `table=True`, se indica que esta clase representa una **tabla real en la base de datos**. Si `table=False` (o no se especifica), la clase se usaría solo como un esquema para validación o transferencia de datos (como en los esquemas Pydantic). Es decir, solo las clases `table=True` generan tablas reales al ejecutar `create_all()`.
- `__tablename__ = "producto"`: Especifica el nombre que tendrá la tabla en la base de datos. En este caso, la tabla se llamará `producto`. Este atributo es opcional en `SQLModel`, pero se usa aquí para tener control explícito sobre el nombre y evitar ambigüedades.
- `Field(...)`:
Se utiliza para añadir metainformación a los atributos del modelo. Aquí se indican:
 - **`primary_key=True`**: el campo es clave primaria.
 - **`nullable=False`**: no se permiten valores nulos (obligatorio en la base de datos).
 - **`unique=True`**: el valor debe ser único (por ejemplo, `sku`).
 - **`index=True`**: se crea un índice para búsquedas más rápidas.

- **default=...**: define un valor por defecto si no se proporciona uno.
- **Optional[str]** (junto con **default=None**) indica que el campo **puede ser nulo** y no es obligatorio.
- **foreign_key="categoria_producto.id"**:
Este campo crea una relación con otra tabla (**categoria_producto**) indicando que **id_categoria** es una clave foránea.

Gracias a esta definición, se genera automáticamente el esquema SQL correspondiente para crear o manipular la tabla **producto**, y además se puede utilizar el modelo como entrada o salida en la API REST sin necesidad de definir estructuras adicionales.

Esquemas: validación y serialización

Los **esquemas** definidos en [tabula-backend/app/schemas](#) se utilizan para controlar qué datos espera o devuelve cada endpoint. Se construyen a partir de **BaseModel** de **Pydantic**, y permiten definir reglas de validación, documentación automática y control de visibilidad de campos.

Un mismo modelo puede tener varios esquemas asociados según el contexto: creación, respuesta, actualización, etc.

Ejemplo de esquema para crear un nuevo usuario:

```

app > schemas > user.py > UserCreate
4
5 class UserBase(BaseModel):
6     """
7     Esquema base para usuarios.
8     - Define los campos comunes a todos los esquemas de usuario.
9     - `EmailStr` valida que el correo tenga formato correcto.
10    """
11
12    nombre: str = Field(
13        ..., min_length=3, max_length=100, description="Nombre del usuario"
14    )
15    email: EmailStr = Field(
16        ..., max_length=100, description="Correo electrónico válido"
17    )
18
19
20 class UserCreate(UserBase):
21     """
22     Esquema para registrar usuarios.
23     - `passwd`: Se exige un mínimo de 8 caracteres.
24     - `rol`: Por defecto, 'usuario'. Solo permite valores válidos.
25     - `activo`: False por defecto, el usuario debe ser activado por un administrador.
26    """
27
28    passwd: str = Field(
29        ...,
30        min_length=8,
31        max_length=255,
32        description="Contraseña segura (mínimo 8 caracteres)",
33    )
34    rol: Optional[str] = Field(
35        default="usuario",
36        pattern="^(usuario|admin)$",
37        description="Rol del usuario (usuario/admin)",
38    )
39    activo: Optional[bool] = Field(
40        default=False, description="Estado activo/inactivo del usuario"
41    )

```

Figura 4: Captura tomada desde VS Code, se muestran los esquemas UserBase y UserCreate

Explicación:

- UserBase(BaseModel) define los campos comunes a todos los esquemas de usuario. Hereda de BaseModel de **Pydantic**, lo que permite aplicar validación automática y generar documentación OpenAPI.
- UserCreate(UserBase) **hereda** de UserBase, y le añade los campos específicos para la creación de usuarios: contraseña (passwd), rol y estado (activo).

Esta separación permite reutilizar el mismo bloque de campos básicos (nombre, email) en otros esquemas como UserUpdate o UserResponse.

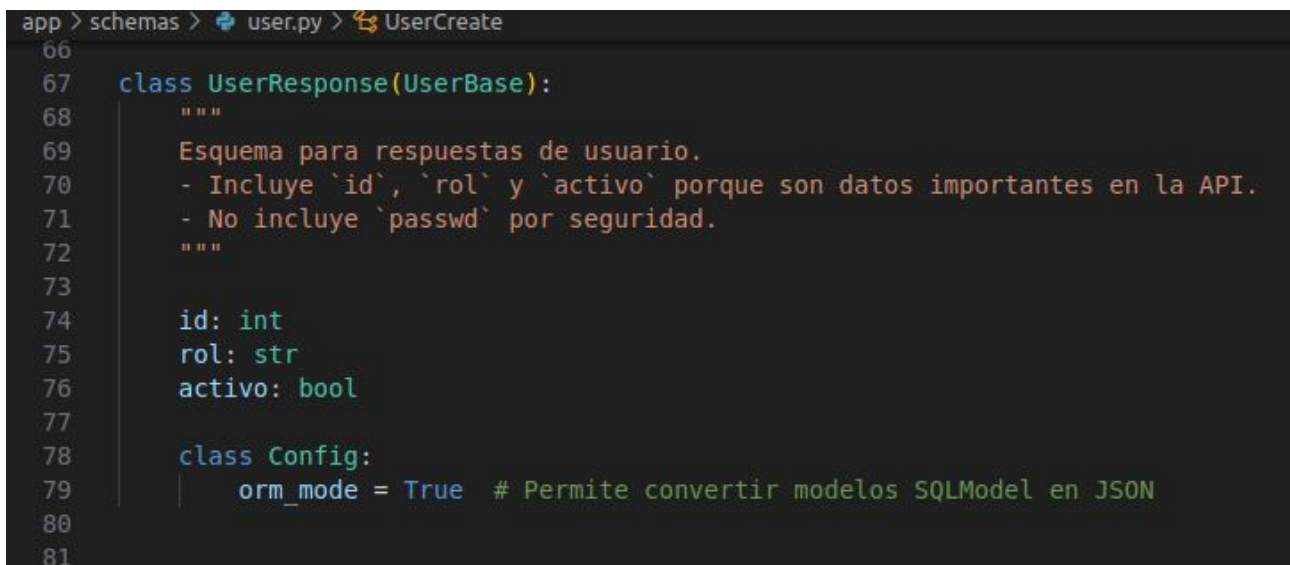
- Field(...) se usa para definir restricciones adicionales:
 - ... significa **campo obligatorio**.
 - min_length y max_length definen la longitud permitida.
 - EmailStr valida automáticamente que el email tenga un formato correcto.
 - pattern asegura que solo se puedan introducir los valores permitidos para el rol: "usuario" o "admin".
- El campo activo está definido como False por defecto, para que un nuevo usuario no esté activo hasta que lo apruebe un administrador.

Esta estructura modular de esquemas permite mantener el código organizado, validado y con una clara separación de responsabilidades.

Este sistema garantiza que:

- Solo los campos permitidos entren en la base de datos.
- Los campos sensibles (como contraseñas) no se devuelvan por accidente.
- Se generen respuestas limpias y tipadas para el frontend.

Los esquemas de respuesta incluyen configuraciones como from_attributes = True o orm_mode = True, que permiten convertir modelos ORM (SQLModel) directamente en respuestas JSON válidas, como podemos ver en la siguiente captura.

A screenshot of a VS Code editor window showing a Python file named 'user.py'. The editor is open to a file named 'UserCreate'. The code defines a class 'UserResponse' that inherits from 'UserBase'. It includes a docstring explaining its purpose and fields. The fields are 'id' (int), 'rol' (str), and 'activo' (bool). There is also an internal class 'Config' with a class attribute 'orm_mode = True' and a comment explaining its purpose.

```
app > schemas > user.py > UserCreate
66
67 class UserResponse(UserBase):
68     """
69     Esquema para respuestas de usuario.
70     - Incluye `id`, `rol` y `activo` porque son datos importantes en la API.
71     - No incluye `passwd` por seguridad.
72     """
73
74     id: int
75     rol: str
76     activo: bool
77
78     class Config:
79         orm_mode = True # Permite convertir modelos SQLAlchemy en JSON
80
81
```

Figura 5: Captura tomada desde VS Code, se muestra el esquema UserResponse

Explicación:

El esquema UserResponse se utiliza para definir la **estructura de los datos devueltos** al cliente cuando se consulta un usuario desde la API (por ejemplo, al hacer login o listar usuarios).

- **Hereda de UserBase**, que contiene los campos comunes: nombre y email.
- Añade los campos adicionales relevantes en una respuesta:
 - id: identificador único del usuario.
 - rol: cadena que indica si el usuario tiene rol usuario o admin.
 - activo: estado actual del usuario (activo/inactivo).
- La clase interna Config establece `orm_mode = True`. Esto es **fundamental** para que FastAPI pueda convertir automáticamente los objetos ORM (como instancias de SQLAlchemy) en diccionarios JSON serializables. Es decir, permite devolver modelos directamente desde consultas SQL sin necesidad de convertirlos manualmente.

Este esquema es el que se utiliza en la mayoría de respuestas relacionadas con usuarios, garantizando que **nunca se incluya el campo passwd** por motivos de seguridad.

Conexión con la base de datos

La conexión a la base de datos se gestiona de forma segura a través del archivo [tabula-backend/app/models/database.py](#). La URL de conexión no se escribe directamente en el código, sino que se carga desde una variable de entorno utilizando la función `get_required_env()` definida en el archivo `getenv.py`, en [app/utils](#).

Esta función asegura que, si falta una variable de entorno esencial, se detenga la ejecución con un mensaje de error, lo que mejora la seguridad y la portabilidad de la aplicación.

```

app > models > database.py > get_db
1  from sqlmodel import SQLModel, create_engine, Session
2  from app.utils.getenv import get_required_env
3
4  # Conectar a la base de datos existente
5  DATABASE_URL = get_required_env("DATABASE_URL")
6
7  engine = create_engine(DATABASE_URL, echo=True)
8
9  SQLModel.metadata.clear()
10
11
12 def get_db():
13     """Obtiene una sesión de la base de datos."""
14     with Session(engine) as session:
15         yield session
16
17
18 def create_db_and_tables():
19     SQLModel.metadata.create_all(engine)
20

```

Figura 6: Captura de `app/models/database.py`, conexión con la base de datos

Explicación:

- `get_required_env("DATABASE_URL")`:
Carga la variable de entorno `DATABASE_URL` (definida en `.env`). Si no existe, lanza una excepción para evitar que la app arranque sin conexión a la base de datos.

- `create_engine(...)`:
Crea el **motor de conexión SQL** a PostgreSQL.
 - `echo=True` muestra en consola todas las sentencias SQL ejecutadas, útil durante el desarrollo para depurar.
- `engine`:
Es el objeto que gestiona todas las conexiones a la base de datos. Se pasa a otras funciones como `Session()` o `create_all()` para trabajar directamente con PostgreSQL.
- Antes de crear las tablas, se llama a `SQLModel.metadata.clear()`, lo cual borra temporalmente la definición de tablas en memoria. Esto puede ser útil en entornos de desarrollo para evitar conflictos si los modelos se cargan varias veces (por ejemplo, con recarga automática), aunque en un entorno productivo esta línea no es estrictamente necesaria.

También se definen dos funciones clave:

- `get_db()`: permite obtener sesiones de base de datos de forma controlada, utilizando generadores (`yield`). Esto permite integrarlo fácilmente como **dependencia** en los endpoints.
- `create_db_and_tables()`: ejecuta la creación de todas las tablas definidas mediante modelos `SQLModel`. Esta función se invoca automáticamente al iniciar la aplicación.

Creación de las tablas al iniciar la app

En el archivo principal [app/main.py](#) se configura la función `lifespan()` con el decorador `@asynccontextmanager`, desde donde se invoca `create_db_and_tables()` al arrancar la aplicación. Esto asegura que la base de datos esté inicializada al iniciar el servidor si se ejecuta en local.

```
20 # Crear la base de datos y las tablas al iniciar la aplicación
21 @asynccontextmanager
22 async def lifespan(app: FastAPI):
23     create_db_and_tables()
24     yield # Aquí se pueden cerrar conexiones u otros recursos
25
```

Figura 7: Función `lifespan()`, para definir acciones que deben ejecutarse al arrancar la aplicación, `main.py`

Explicación:

- `@asynccontextmanager`:
Es un decorador que define un **bloque de código que se ejecuta al iniciar y cerrar la app**.
- `lifespan(app)` se pasa a `FastAPI()` para definir **acciones que deben ejecutarse al arrancar la aplicación**.
- Dentro de la función, se llama a `create_db_and_tables()`, lo que permite crear todas las tablas en la base de datos **al inicio de la app si no existen**.
- `yield` marca el punto en que FastAPI continuará con su ciclo de vida normal (carga de rutas, etc.). Después de `yield`, podrían ir acciones de limpieza si fuera necesario (como cerrar conexiones).

Subtarea 2.4.2: Implementación de autenticación con JWT

Para garantizar la seguridad en el acceso a los endpoints, se implementó un sistema de **autenticación con JWT (JSON Web Tokens)**, siguiendo las [recomendaciones oficiales de FastAPI^{\[7\]}](#). Este sistema permite identificar de forma segura a los usuarios y controlar el acceso a ciertas rutas según su rol (usuario o administrador).

La implementación se estructuró en dos componentes principales:

1. La lógica de autenticación y generación/verificación de tokens, definida en el archivo [utils/authentication.py](#).

2. Las rutas públicas de autenticación (/auth/login, /auth/refresh, etc.), definidas en [routers/auth.py](#).

Previo a eso, se estableció la estructura para montar las rutas correctamente desde main.py.

Montaje de rutas en main.py

El archivo [main.py](#) es el punto de entrada de la aplicación. En él se incluye cada módulo de rutas mediante `app.include_router(...)`, organizando el backend de forma modular y legible.

```
24 app = FastAPI(lifespan=lifespan)
25
26 # Configuración CORS segura para cookies
27 app.add_middleware(
28     CORSMiddleware,
29     allow_origins=["http://localhost:5173", "https://tabula-frontend.vercel.app"],
30     allow_credentials=True,
31     allow_methods=["*"],
32     allow_headers=["*"],
33 )
34
35 # Incluir routers
36 app.include_router(auth.router)
37 app.include_router(users.router)
38 app.include_router(products.router)
39 app.include_router(movements.router)
40 app.include_router(warehouses.router)
41 app.include_router(stock.router)
42 app.include_router(product_categories.router)
43 # Websocket
44 app.include_router(websocket_router)
45
46
47 @app.get("/")
48 def read_root():
49     return {"message": "API funcionando correctamente"}
50
```

Figura 8: Captura de pantalla desde VS Code del fichero main.py, inclusión de routers y definición de la ruta "/", ruta raíz de la aplicación.

Cada archivo de rutas (por ejemplo auth.py) define su propio prefijo, lo que permite agrupar los endpoints relacionados bajo una misma URL base. En el caso de autenticación, el prefijo utilizado es /auth.

En la parte superior de main.py se incluye y configura el middleware **CORS (Cross-Origin Resource Sharing)**.

¿Por qué es necesario esto?

- En una arquitectura típica **frontend-backend separada**, las peticiones que hace el frontend hacia la API FastAPI son **cross-origin** (es decir, entre distintos dominios u orígenes).
- Los navegadores, por motivos de seguridad, **bloquean por defecto** este tipo de peticiones, a menos que el backend incluya un middleware que permita explícitamente el acceso desde esos orígenes.

Explicación de cada parámetro:

- `allow_origins`: define los orígenes permitidos. Durante el desarrollo, se usa `localhost`; en producción, el dominio del frontend (en este caso, Vercel).
- `allow_credentials=True`: permite enviar y recibir cookies o encabezados de autenticación (como los tokens JWT).
- `allow_methods=["*"]`: permite cualquier tipo de petición (GET, POST, etc.).
- `allow_headers=["*"]`: permite cualquier encabezado personalizado necesario para las peticiones.

Esta configuración es esencial para que el sistema de autenticación funcione correctamente, especialmente porque el **refresh token se envía como una cookie `HttpOnly`**, que solo se puede transmitir si CORS está bien configurado.

Esquema de autenticación JWT

El sistema de autenticación implementado se basa en **JWT (JSON Web Tokens)**^[8], una técnica común en APIs modernas que permite autenticar usuarios de forma **segura, sin necesidad de mantener sesiones en el servidor**.

Cuando un usuario inicia sesión correctamente, el servidor genera un **token JWT firmado** que incluye en su interior información relevante (como su ID y rol). Este token actúa como una "credencial portable", que el cliente (navegador) enviará en cada petición posterior para demostrar su identidad.

Los JWT son **autoverificables**: al recibir uno, el servidor puede comprobar su validez sin necesidad de hacer una consulta adicional a la base de datos, ya que el token está firmado con una clave secreta.

En este proyecto, el sistema JWT implementado consta de dos tipos de tokens:

- **Access token**: de duración corta (por defecto, 30 minutos). Se utiliza para acceder a los endpoints protegidos.
- **Refresh token**: de duración más larga (por defecto, 7 días). Permite obtener un nuevo access token sin tener que volver a hacer login.

Flujo básico:

1. El usuario inicia sesión enviando su email y contraseña a `/auth/login`.
2. Si las credenciales son correctas, se generan dos tokens:
 - El access token se devuelve como JSON.
 - El refresh token se guarda en una **cookie HttpOnly**, no accesible desde JavaScript (mayor seguridad).
3. En cada solicitud posterior, el access token se envía en el encabezado `Authorization (Bearer <token>)`.
4. Si el token caduca, el frontend puede usar `/auth/refresh` para obtener uno nuevo usando la cookie del refresh token.
5. Para cerrar sesión, se llama al endpoint `/auth/logout`, que borra la cookie del refresh token.

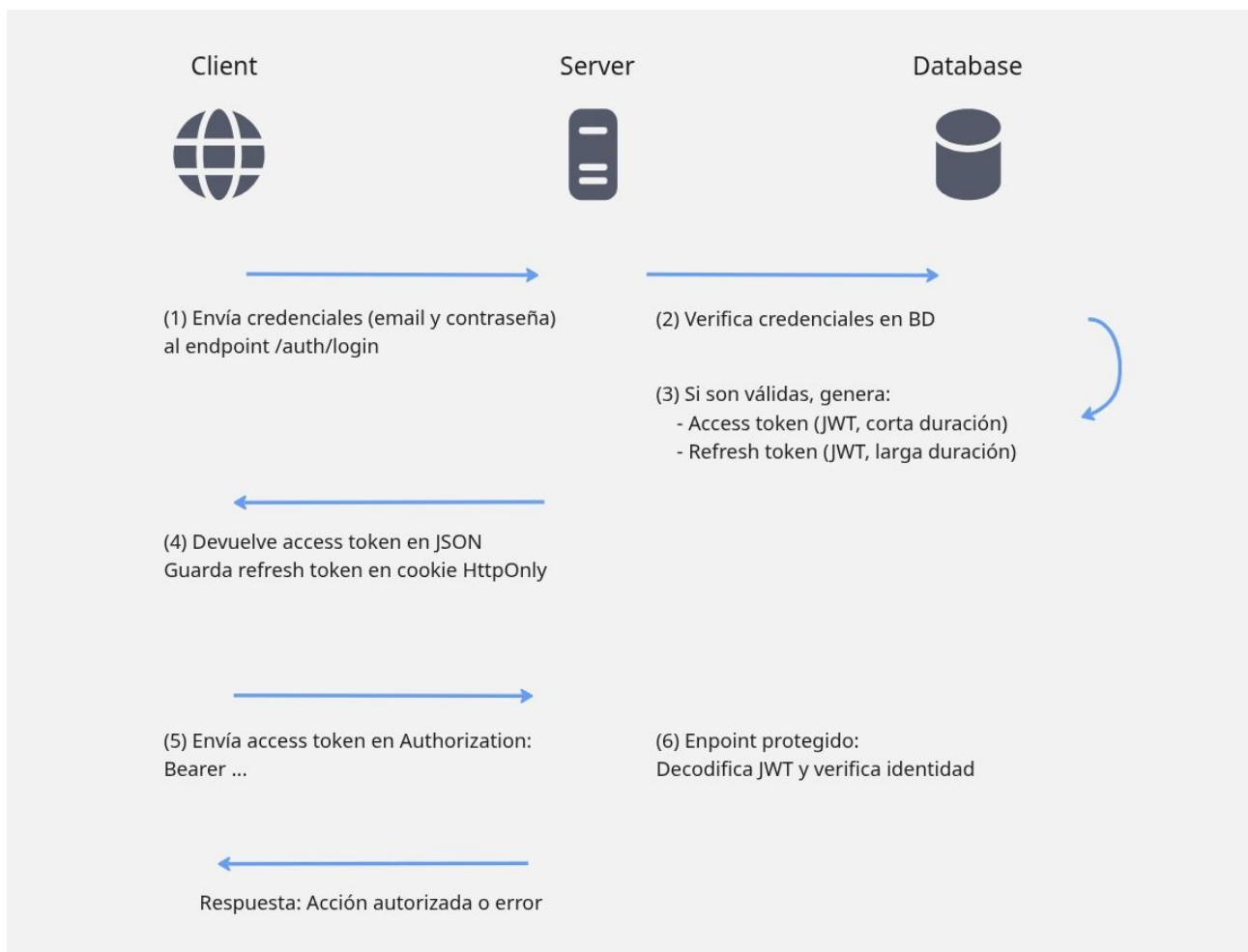


Figura 9: Esquema de autenticación inicial mediante JWT, creación propia (miro.com)

Lógica de autenticación ([utils/authentication.py](#))

Este archivo contiene las funciones que manejan la autenticación JWT y el hash de contraseñas.

Se definen:

- **hash_password(...)**: para guardar contraseñas cifradas (usando bcrypt).
- **verify_password(...)**: para verificar contraseñas al hacer login.
- **create_access_token(...)**: genera un token firmado con fecha de expiración.
- **decode_access_token(...)**: verifica y decodifica un token. Si está expirado o es inválido, lanza una excepción.

Definición de rutas en FastAPI

En FastAPI, cada **endpoint** se define como una función asociada a un método HTTP (GET, POST, PUT, DELETE, etc.) utilizando decoradores como `@router.get(...)` o `@router.post(...)`. Estas funciones se agrupan por temática en archivos dentro de la carpeta `routers/`, y se montan en `main.py` con un prefijo que organiza las URLs.

Cada ruta puede aceptar parámetros, dependencias (como acceso a la base de datos o autenticación), y devuelve una respuesta estructurada usando esquemas Pydantic.

```
101  ### LOGIN DE USUARIO ###
102  @router.post("/login")
103  def login(
104      response: Response,
105      form_data: OAuth2PasswordRequestForm = Depends(),
106      db: Session = Depends(get_db),
107  ):
108      """Autentica al usuario y genera un token JWT."""
```

Figura 10: Captura desde VS Code, extracto de la cabecera de la definición del endpoint `/login`

Esta forma modular permite mantener el código limpio, reutilizable y fácilmente testeable.

Rutas de autenticación ([routers/auth.py](#))

En este archivo se definen los endpoints que gestionan el flujo de autenticación, responsables de gestionar el ciclo de autenticación de los usuarios. Esto incluye el registro, el login, la validación de tokens, el refresco de sesión, el acceso al perfil del usuario autenticado y el cierre de sesión. Las rutas de autenticación son las siguientes:

/auth/registro (POST)

Este endpoint permite a nuevos usuarios registrarse en el sistema. Valida que el correo electrónico no esté ya en uso y, si todo es correcto:

- Guarda al usuario en la base de datos con contraseña **encriptada con bcrypt**.
- Asigna automáticamente el rol "usuario" y lo marca como **inactivo** por defecto (requiere activación manual por parte de un administrador).
- Devuelve los datos del nuevo usuario (sin la contraseña, gracias al esquema `UserResponse`).

/auth/login (POST)

El login se realiza mediante un formulario (OAuth2PasswordRequestForm) que recibe el email como username y la contraseña como password. Si las credenciales son correctas y el usuario está activo:

- Se genera un **access token** (JWT) con datos como su ID y rol.
- Se genera un **refresh token**, que se guarda en una **cookie segura HttpOnly**.
- Se devuelve el access token en la respuesta para su uso en futuras solicitudes autenticadas.

La cookie del refresh token se configura con:

- httponly=True (no accesible desde JS),
- secure=True (solo en HTTPS),
- samesite="none" (para permitir uso cross-origin desde el frontend),
- y path="/auth/refresh" para limitar su uso al endpoint correspondiente.

/auth/perfil (GET)

Este endpoint está protegido y devuelve los datos del usuario autenticado. Se basa en el token de acceso enviado en el encabezado Authorization (Bearer token), el cual se decodifica para identificar al usuario y devolver su información. El esquema de respuesta usado es UserResponse, que excluye automáticamente datos sensibles como la contraseña.

/auth/refresh (POST)

Permite obtener un **nuevo access token** a partir del refresh token almacenado en la cookie. Se usa cuando el access token ha caducado, para mantener al usuario logueado sin pedirle que vuelva a iniciar sesión. El endpoint verifica el refresh token y, si es válido, emite un nuevo access token.

/auth/verify-password (POST)

Este endpoint permite que un usuario autenticado verifique su contraseña actual. Se usa, por ejemplo, antes de permitir que el usuario edite datos sensibles (como su

nombre o correo). Se compara la contraseña introducida con la almacenada (encriptada) y se devuelve un mensaje de confirmación o error.

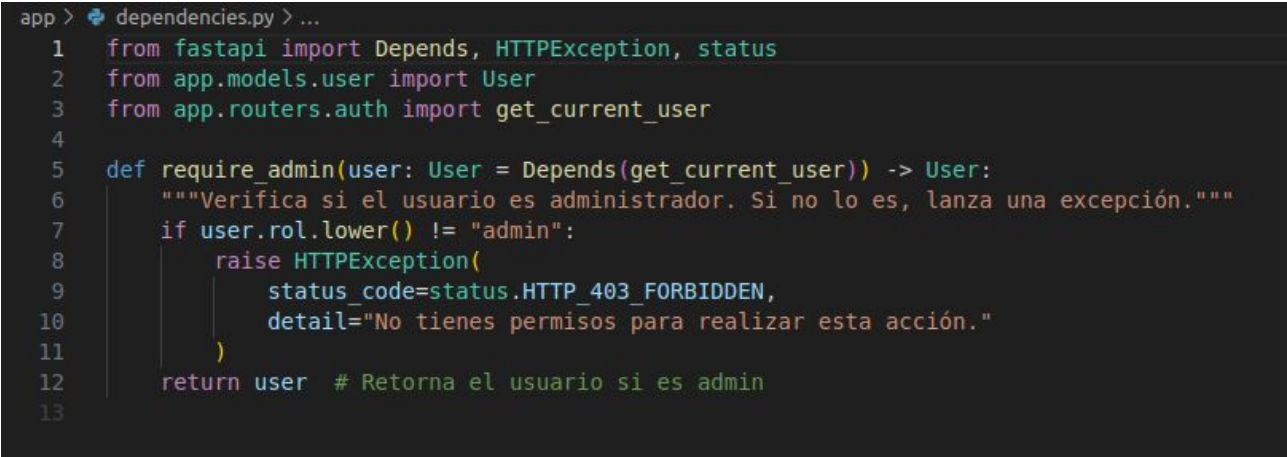
/auth/logout (POST)

Cierra la sesión actual eliminando la cookie del refresh token. Esto impide que el usuario pueda generar un nuevo access token hasta que vuelva a iniciar sesión.

Todos estos endpoints utilizan dependencias (Depends) y manejo de excepciones estructurado para asegurar respuestas claras y seguras. El middleware CORS, previamente configurado en main.py, permite que estas rutas funcionen correctamente incluso si el frontend se aloja en otro dominio.

Seguridad adicional: uso de roles

En [app/dependencies.py](#) se define la función `require_admin(...)` para proteger endpoints que solo deben ser accesibles por administradores.

A screenshot of a VS Code editor window showing a Python file named dependencies.py. The code defines a function require_admin that takes a user object as an argument. It checks if the user's role is 'admin'. If not, it raises an HTTPException with status code 403 and a message. If the user is an admin, it returns the user object. The code is as follows:

```
app > dependencies.py > ...
1  from fastapi import Depends, HTTPException, status
2  from app.models.user import User
3  from app.routers.auth import get_current_user
4
5  def require_admin(user: User = Depends(get_current_user)) -> User:
6      """Verifica si el usuario es administrador. Si no lo es, lanza una excepción."""
7      if user.rol.lower() != "admin":
8          raise HTTPException(
9              status_code=status.HTTP_403_FORBIDDEN,
10             detail="No tienes permisos para realizar esta acción."
11         )
12     return user # Retorna el usuario si es admin
13
```

Figura 11: Captura de VS Code, función `require_admin` en `dependencies.py`

Esto permite declarar un endpoint así:

```
22 @router.get("/", response_model=PaginatedUserResponse)
23 def get_users(
24     db: Session = Depends(get_db),
25     current_user: User = Depends(require_admin),
26     limit: int = Query(10, ge=1, le=1000),
27     offset: int = Query(0, ge=0),
28     search: str = Query(None),
29     estado: bool = Query(None),
30 ):

```

Figura 12: Captura de VS Code, uso de dependencia `require_admin` en endpoint GET `/users`, `get_users()`

Subtarea 2.4.3: Diseño del resto de endpoints y CRUDs

Tras definir la autenticación y el sistema de validación de usuarios, se desarrollaron los distintos endpoints RESTful que permiten gestionar los recursos principales de la aplicación: usuarios, productos, almacenes, categorías, movimientos y stock. La mayoría de estos endpoints implementan operaciones CRUD (crear, leer, actualizar, eliminar), siguiendo las convenciones REST y los modelos de datos definidos previamente.

Las rutas se encuentran organizadas en módulos separados dentro de la carpeta `routes/`, y se documentan automáticamente a través de Swagger UI y ReDoc.

Una vez se tenga la API corriendo localmente, se puede acceder a la documentación desde:

- <http://localhost:8000/docs>→ Interfaz Swagger (explorable y testeable)
- <http://localhost:8000/redoc>→ Redoc (alternativa más estructurada)

Cuando el backend esté desplegado, estos mismos endpoints estarán disponibles bajo la URL pública del servidor, para este proyecto estarán disponibles hasta el 24/05/25 en:

- <https://tabula-backend-n86c.onrender.com/docs>
- <https://tabula-backend-n86c.onrender.com/redoc>

Desde estas interfaces puedes consultar, probar y entender todos los endpoints disponibles en tiempo real.

Endpoints de usuarios (usuarios.py)

El archivo [users.py](#) define las rutas que permiten a los administradores gestionar a los usuarios registrados en el sistema. Incluye operaciones de listado, creación, actualización, activación/desactivación en lote, y eliminación. Algunos endpoints también son accesibles por los propios usuarios para consultar o actualizar su perfil.

Principales funcionalidades:

- **GET /usuarios/**

Lista todos los usuarios registrados. Solo accesible para administradores. Incluye filtros por estado (activo/inactivo) y búsqueda por nombre o correo.

- **GET /usuarios/{id}**

Devuelve los datos de un usuario específico.

- Un administrador puede consultar cualquier usuario.
- Un usuario normal solo puede consultar su propio perfil.

- **POST /usuarios/**

Crea un nuevo usuario.

- Solo los administradores pueden crear usuarios.
- Valida que el correo no esté repetido y que el rol sea válido (usuario o admin).

- **PUT /usuarios/{id}**

Permite actualizar los datos de un usuario.

- El propio usuario puede modificar su nombre, correo o contraseña.
- Solo un administrador puede cambiar el rol o el estado (activo).
- Se valida que el nuevo correo no esté en uso por otro usuario.

- **PUT /usuarios/estado-multiple**

Activa o desactiva varios usuarios a la vez.

- Solo administradores pueden realizar esta acción.
- Se evita desactivar al usuario autenticado a sí mismo.

- **DELETE /usuarios/{id}**

Elimina un usuario permanentemente, **solo si no tiene movimientos registrados**.

- Solo accesible por administradores.
- Si el usuario ha participado en movimientos, la eliminación se bloquea para preservar la integridad histórica del sistema.

Estos endpoints están protegidos mediante el sistema de roles y autenticación JWT implementado previamente. Se reutilizan funciones de validación como **is_admin_user()** y se aplican reglas de negocio como la no eliminación de usuarios con historial.

Endpoints de productos (products.py)

El archivo [products.py](#) contiene los endpoints REST que permiten gestionar el catálogo de productos en el sistema. La mayoría de las operaciones están restringidas a usuarios administradores, salvo la consulta de productos activos, que puede ser realizada por cualquier usuario autenticado.

Los productos pueden clasificarse por categoría, tener estado activo/inactivo, y están asociados a registros de stock y movimientos. También se valida que no se puedan eliminar productos con stock o movimientos asociados.

Principales funcionalidades:

- **GET /productos/**

Devuelve un listado paginado de productos, con opciones de filtro por:

- Nombre o SKU (search)
 - Categoría (id_categoria)
 - Estado (activo/inactivo, solo visible para administradores)
- Los usuarios normales solo ven productos activos. El listado incluye también el nombre de la categoría asociada a cada producto.

- **GET /productos/{id}**

Devuelve los datos de un producto específico, incluyendo su categoría.

- Los usuarios normales solo pueden ver productos activos.
- Los administradores pueden ver cualquier producto, incluso si está inactivo.

- **POST /productos/**

Crea un nuevo producto. Solo accesible a administradores.

- Se valida que el SKU no esté duplicado.
- Se verifica que la categoría especificada exista.

- **PUT /productos/{id}**

Actualiza los datos de un producto.

- Cualquier usuario puede editar información general (nombre, descripción, categoría).
- Solo los administradores pueden cambiar el estado activo.
- Se valida que el nuevo SKU no esté en uso por otro producto.

- **PUT /productos/estado-multiple**

Permite activar o desactivar varios productos a la vez (solo administradores).

- Si un producto tiene stock registrado, **no se permite desactivarlo**.

- **DELETE /productos/{id}**

Elimina un producto, siempre que **no tenga movimientos asociados**.

- Acción reservada a administradores.
- Se verifica que el producto exista y se obtienen datos de categoría para la respuesta.

Todos los endpoints realizan validaciones y manejo de errores frente a la base de datos. Se implementa paginación, búsquedas flexibles y respuesta enriquecida con información de categoría cuando es relevante.

Endpoints de movimientos (movements.py)

El archivo [movements.py](#) define los endpoints que permiten **registrar y consultar movimientos de stock**, junto con sus líneas. Cada movimiento representa una operación de entrada o salida de productos en un almacén. También se incluyen endpoints adicionales para la generación de gráficas y resúmenes estadísticos del sistema.

Principales funcionalidades:

- **GET /movimientos/**

Devuelve una lista paginada de movimientos. Incluye filtros por:

- Usuario (usuario_id, solo admins)
- Tipo (entrada o salida)
- Rango de fechas (fecha_desde, fecha_hasta)
- Nombre del usuario (search). Los usuarios normales solo ven sus propios movimientos. La respuesta incluye las líneas asociadas y el nombre del usuario.

- **GET /movimientos/{id_mov}**

Devuelve los detalles completos de un movimiento, incluyendo sus líneas.

- Los usuarios normales solo pueden consultar movimientos propios.
- Los administradores pueden consultar cualquier movimiento.

- **POST /movimientos/**

Registra un nuevo movimiento de tipo entrada o salida, junto con sus líneas, en una única petición.

- Se verifica que el usuario tenga permisos para registrar el movimiento (solo admins pueden registrar movimientos para otros usuarios).
- Se valida que todos los productos y almacenes implicados estén activos.
- Se impide registrar productos con fecha de caducidad vencida en movimientos de entrada.

- Se limita el número máximo de líneas por movimiento (100).
- Tras completarse, se emite un **mensaje por WebSocket** notificando la creación del nuevo movimiento a todos los clientes conectados.

- **GET /movimientos/{id_mov}/lineas**

Devuelve una lista paginada de las líneas de un movimiento, incluyendo el **nombre del producto y del almacén** asociados.

- Aplica validaciones similares al acceso a movimientos.

- **GET /movimientos/last-year**

Devuelve una lista de movimientos registrados en el último año.

- Utilizado para construir **gráficas temporales**.
- Los usuarios normales solo reciben sus propios datos.

- **GET /movimientos/resumen/tipo**

Devuelve un **resumen de movimientos agrupado por tipo** (entrada/salida).

- Se utiliza para gráficos de tipo circular o resumen general.
- Los usuarios normales solo reciben el resumen de sus movimientos.

Este módulo contiene validaciones exhaustivas sobre productos, almacenes, fechas, integridad de stock y permisos, siendo uno de los componentes centrales de la lógica del sistema.

Además, es el único que incorpora **integración directa con WebSocket**, permitiendo la notificación en tiempo real al frontend cuando se registra un nuevo movimiento.

Endpoints de almacenes (warehouses.py)

El archivo [warehouses.py](#) define los endpoints REST para la **gestión de almacenes** dentro del sistema. Los almacenes son esenciales para organizar el stock, registrar movimientos y consultar inventario por ubicación. Estos endpoints permiten crear, editar, activar/desactivar, eliminar y listar almacenes, aplicando restricciones según el rol del usuario y el estado de ocupación del almacén.

Principales funcionalidades:

- **GET /almacenes/**

Devuelve un listado paginado de almacenes.

- Cualquier usuario autenticado puede consultar esta lista.
- Se puede filtrar por texto (search) y estado (activo/inactivo).
- Los resultados están ordenados por descripción.

- **GET /almacenes/{codigo}**

Devuelve los datos de un almacén específico.

- Los administradores pueden acceder a cualquier almacén, activo o no.
- Los usuarios normales **no pueden consultar almacenes inactivos**.

- **POST /almacenes/**

Crea un nuevo almacén.

- Solo accesible a administradores.
- Se crea con el estado activo=True por defecto.
- La descripción debe ser única y válida.

- **PUT /almacenes/{codigo}**

Permite actualizar la descripción o el estado de un almacén.

- Solo los administradores pueden modificar.
- Si se intenta desactivar un almacén con productos dentro, la operación se bloquea para evitar inconsistencias.

- **PUT /almacenes/estado-multiple**

Activa o desactiva varios almacenes a la vez (solo administradores).

- Si un almacén contiene stock, **no se puede desactivar**.
- La respuesta incluye el número de almacenes modificados y omitidos.

- **DELETE /almacenes/{codigo}**

Elimina un almacén permanentemente.

- Solo permitido si el almacén **no tiene movimientos registrados**.
- Requiere permisos de administrador.
- Si existen movimientos en `movimientos_lineas` asociados al almacén, se impide su eliminación.

Estos endpoints protegen la integridad de la información y evitan operaciones potencialmente destructivas si el almacén tiene productos registrados o historial de movimientos.

Se aplican múltiples validaciones personalizadas según el tipo de operación y el rol del usuario autenticado.

Endpoints de categorías de producto (`product_categories.py`)

El archivo [`product\_categories.py`](#) contiene los endpoints necesarios para gestionar las **categorías asociadas a los productos**. Estas categorías permiten agrupar productos por tipo (alimentación, limpieza, etc.) y facilitan tanto la organización del catálogo como los filtros en el frontend.

Todos los endpoints de esta sección están protegidos: **solo los administradores pueden crear, editar o eliminar categorías**, aunque su consulta es pública dentro de otros endpoints (por ejemplo, al listar productos).

Principales funcionalidades:

- **GET /categorias/**

Devuelve una lista paginada de todas las categorías registradas.

- Incluye el total de categorías disponibles.
- La respuesta está ordenada alfabéticamente por nombre.
- Este endpoint se usa principalmente para mostrar categorías en formularios o filtros del frontend.

- **POST /categorias/**

Crea una nueva categoría (solo administradores).

- El nombre se normaliza para evitar duplicados con diferencias de mayúsculas/minúsculas.
- Se evita registrar una categoría si ya existe otra con el mismo nombre (normalizado).

- **PUT /categorias/{id}**

Actualiza el nombre de una categoría existente (solo administradores).

- Valida que no exista otra categoría con el nuevo nombre.
- Si el ID no corresponde a ninguna categoría, se devuelve error 404.

- **DELETE /categorias/{id}**

Elimina una categoría si **no hay productos asociados**.

- Antes de eliminar, se verifica que ningún producto tenga esa categoría asignada.
- En caso contrario, se devuelve un error informando que no puede eliminarse por estar en uso.

Este módulo es fundamental para mantener la consistencia semántica de los productos y garantizar que las categorías sean únicas, válidas y editables únicamente por administradores.

Las operaciones están orientadas a facilitar una gestión sencilla pero controlada del catálogo.

Endpoints de stock (stock.py)

A diferencia del resto de módulos, el archivo [stock.py](#) no contiene operaciones CRUD. Todos los endpoints son de tipo **GET** y están diseñados exclusivamente para **consultar el estado actual del inventario**, tanto a nivel general como filtrado por producto, almacén o lote.

Estos endpoints permiten construir las distintas **vistas de stock** del sistema y alimentar las **gráficas del dashboard** que muestran el estado del inventario.

Principales funcionalidades:

- **GET /stock/producto/{codigo_producto}**

Devuelve el stock total de un producto en todos los almacenes.

- Incluye cantidad, lote y fecha de caducidad.
- Utilizado en la vista de stock por producto.
- Se excluyen entradas con cantidad cero.

- **GET /stock/almacen/{codigo_almacen}**

Devuelve el stock total en un almacén determinado, agrupado por producto.

- Se utiliza en la vista de stock por almacén.
- Permite mostrar todos los productos con stock positivo en ese almacén.

- **GET /stock/almacen/{codigo_almacen}/producto/{codigo_producto}**

Devuelve el stock detallado de un **producto concreto en un almacén específico**, incluyendo lote y fecha de caducidad.

- Útil para ver qué lotes están disponibles de un producto dentro de un almacén.
- Sirve como base para generar gráficas de estado por lote.

- **GET /stock/caducidad**

Devuelve productos próximos a caducar en un rango de fechas.

- Permite pasar como parámetros desde y hasta (fechas).
- Se usa para detectar productos sensibles o lotes a punto de vencer.
- Fundamental para construir la **vista de caducidades** y su gráfica.

- **GET /stock/producto/{codigo_producto}/historial**

Devuelve el historial de movimientos de un producto determinado.

- La respuesta se puede paginar.

- Útil para representar gráficamente la evolución del stock a lo largo del tiempo.
- Usado también para trazar movimientos pasados en la vista de detalle.
- **GET /stock/almacen/{codigo_almacen}/historial**

Similar al anterior, pero centrado en un almacén concreto.

- Devuelve el historial de entradas y salidas del almacén.
- Sirve para generar gráficas temporales sobre la actividad del almacén.

Estos endpoints están diseñados para integrarse con el frontend, devolviendo únicamente la información relevante para las **vistas operativas y gráficas** del sistema. Ninguno de ellos modifica el estado de la base de datos, por lo que pueden usarse con seguridad desde componentes en tiempo real.

Pruebas con Thunder Client

Durante el desarrollo del backend, se realizaron pruebas continuas de todos los endpoints utilizando la extensión **Thunder Client** en Visual Studio Code. Esto permitió validar de forma rápida y visual que las rutas respondían correctamente, que la lógica de autenticación funcionaba como se esperaba y que las validaciones de entrada y respuesta eran coherentes. Esta herramienta fue especialmente útil para detectar errores tempranos y verificar tanto peticiones autorizadas como no autorizadas.

Duración

Tarea/Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 2: Desarrollo del Backend (Base de Datos y API REST)		24/02/2025	16/03/2025
Subtarea 2.1: Configuración del entorno de desarrollo	1	24/02/2025	25/02/2025
Subtarea 2.2: Diseño e implementación de la base de datos		26/02/2025	28/02/2025
2.2.1: Configuración inicial	2	26/02/2025	26/02/2025
2.2.2: Modelo relacional y entidades	5	26/02/2025	27/02/2025
2.2.3: Trigger automático de stock	3	28/02/2025	28/02/2025
Subtarea 2.3: Estructuración del backend y preparación entorno		28/02/2025	01/03/2025
2.3.1: Preparación del entorno virtual	0,5	28/02/2025	28/02/2025
2.3.2: Instalación de dependencias	0,5	01/03/2025	01/03/2025
Subtarea 2.4: Desarrollo de la API REST con FastAPI		02/03/2025	16/03/2025
2.4.1: Modelos, esquemas y conexión con la base de datos	17	02/03/2025	06/03/2025
2.4.2: Implementación de autenticación con JWT	13	07/03/2025	09/03/2025
2.4.3: Diseño del resto de endpoints y CRUDs	18	10/03/2025	16/03/2025
TOTAL	60		

Tarea 3. Desarrollo del Frontend (React y Tailwind)

Subtarea 3.1: Formación y configuración del entorno frontend

Subtarea 3.1.1 Formación

Antes de iniciar el desarrollo de la interfaz de usuario, se llevó a cabo una fase de **formación intensiva en React**, con el objetivo de adquirir los conocimientos necesarios para construir componentes reutilizables, gestionar el estado, implementar rutas dinámicas y comunicarse con APIs externas.

Esta etapa se extendió aproximadamente durante **una semana**, combinando teoría y práctica, y permitió sentar una base sólida para el desarrollo del frontend con herramientas modernas.

Arquitectura SPA y fundamentos de React

React es una biblioteca de JavaScript diseñada para construir interfaces de usuario interactivas y eficientes. En lugar de renderizar páginas completas desde el servidor en cada navegación, React adopta el modelo de **Single Page Application (SPA)**, donde toda la aplicación se carga una vez y el contenido se actualiza dinámicamente en el navegador sin recargar la página completa.

Este enfoque ofrece ventajas clave:

- **Velocidad:** las transiciones entre vistas son rápidas, ya que no se solicita una nueva página al servidor.
- **Interactividad:** los componentes se actualizan de forma reactiva ante cambios de datos o eventos.
- **Escalabilidad:** se pueden componer interfaces complejas a partir de pequeños componentes reutilizables.

React se basa en conceptos como:

- **JSX:** una extensión de JavaScript que permite escribir HTML dentro del código JS.
- **Componentes:** funciones que encapsulan lógica y presentación.

- **Hooks** como `useState` o `useEffect` para manejar estado, efectos secundarios y lógica compartida.
- **Enrutamiento dinámico** con **react-router-dom** para gestionar la navegación sin recargar la página.

Una característica destacada de React es su enfoque **declarativo**: en lugar de decirle al navegador cómo modificar la interfaz paso a paso, se describe cómo debería verse en función del estado actual, y React se encarga de aplicar los cambios necesarios.

Subtarea 3.1.2: Configuración del entorno de desarrollo

Para desarrollar el frontend de la aplicación se utilizó **Vite**, una herramienta moderna de construcción (build tool) que reemplaza a sistemas tradicionales como Webpack. Vite destaca por su **velocidad de arranque** y su sistema de **recarga instantánea**.

En lugar de empaquetar toda la aplicación antes de servirla, Vite utiliza **servidores basados en ES modules** y compila los archivos bajo demanda, lo que acelera significativamente el proceso de desarrollo.

En una aplicación React creada con Vite, todo el contenido se monta dinámicamente sobre un único archivo HTML, el cual actúa como **puerta de entrada de la SPA**. En este archivo, solo se declara una etiqueta `<div>` vacía con un `id="root"`: es ahí donde React renderiza toda la aplicación.

A continuación se muestra el contenido del archivo **tabula-frontend/[index.html](#)** para ilustrar esta idea:

A screenshot of a code editor showing the content of index.html. The code is as follows:

```
<? index.html > <? html
1  <!doctype html>
2  <html lang="en">
3
4  <head>
5    <meta charset="UTF-8" />
6    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7    <title>Tábula</title>
8  </head>
9
10 <body>
11   <div id="root"></div>
12   <script type="module" src="/src/main.jsx"></script>
13 </body>
14
15 </html>
```

Figura 13: Captura de index.html, desde VS Code, punto de montaje de la aplicación

El elemento `<div id="root"></div>` es el **punto de montaje** de toda la aplicación. Cuando se ejecuta el proyecto, React inyecta ahí todos los componentes, vistas, navegación y lógica de la aplicación, gestionando los cambios de forma interna sin recargar la página.

Más adelante, veremos como en el archivo main.jsx, se especifica precisamente que React debe renderizar el componente principal (`<App />`) dentro de ese div.

Pasos seguidos para crear el proyecto

1. Instalación de Node.js

Se parte de una instalación actualizada de **Node.js**, que incluye npm (Node Package Manager) para gestionar dependencias.

2. Creación del proyecto con Vite

```
npm create vite@latest
```

Durante este proceso se seleccionó el framework React y se eligió el formato con JavaScript (en lugar de TypeScript).

3. Acceso a la carpeta del proyecto

El proyecto generado se ubicó en una carpeta llamada tabula-frontend:

```
cd tabula-frontend
```

4. Instalación de dependencias

Una vez dentro del proyecto, se instalaron las dependencias base:

```
npm install
```

5. Inicio del entorno de desarrollo

Para lanzar el servidor de desarrollo:

```
npm run dev
```

Esto inicia la aplicación en localhost con recarga automática cada vez que se guarda un archivo.

El archivo [package.json](#) generado en este proceso define las dependencias principales del proyecto y los scripts disponibles para desarrollo (dev), build de producción (build), linting (lint) y previsualización (preview).

El árbol de directorios inicial se generó automáticamente al crear el proyecto, y posteriormente se adaptó a las necesidades del mismo:

tabula-frontend

```
|— eslint.config.js
|— index.html ← Punto de montaje de la aplicación
|— LICENSE
|— package.json ← Dependencias
|— package-lock.json
|— README.md
|— src
| |— api/ ← Peticiones HTTP organizadas por entidad
| |— App.jsx ← Componente raíz
| |— components/ ← Componentes reutilizables
| |— context/ ← Contextos como autenticación
| |— hooks/ ← Hooks personalizados
| |— index.css ← Estilos base con Tailwind
| |— main.jsx ← Punto de entrada de la app
| |— pages/ ← Vistas principales por ruta
| |— router/ ← Definición de rutas y protección por roles
| |— utils/ ← Funciones auxiliares y utilidades
|— vercel.json
|— vite.config.js
```

Esta estructura permite separar claramente los distintos aspectos de la aplicación, mejorar la escalabilidad y facilitar el mantenimiento.

Dependencias principales

Además de React y React Router, el proyecto incluye varias librerías para mejorar la experiencia de desarrollo y usuario. Algunas de las más relevantes son:

- **tailwindcss**: sistema de clases utilitarias para el diseño rápido y responsivo de la interfaz.
- **react-router-dom**: para navegación SPA con rutas protegidas.
- **react-select**: selectores personalizados con carga dinámica.

- **recharts**: librería de gráficos para visualización de datos.
- **jwt-decode**: para analizar y validar tokens JWT en el cliente.
- **papaparse**: para exportación de datos en formato CSV.

Las dependencias completas se detallan en el archivo **package.json**, que también contiene los scripts necesarios para iniciar (npm run dev), construir (npm run build) y previsualizar (npm run preview) la aplicación.

Subtarea 3.2: Diseño general de la interfaz de usuario (UI)

El diseño de la interfaz de la aplicación **Tábula** se planificó con el objetivo de combinar claridad visual con facilidad de navegación. Se buscó un equilibrio entre estética moderna y enfoque práctico, adaptado al contexto de una herramienta de gestión.

Se optó por una **interfaz responsiva**, optimizada para su uso en dispositivos de escritorio, pero preparada para ajustarse correctamente a resoluciones más reducidas. El uso de tipografías legibles, colores contrastados y tamaños de botón adecuados facilita la interacción.

Toda la UI se construyó mediante clases de **Tailwind CSS**, un framework de utilidades CSS que permite componer estilos directamente en los componentes mediante clases semánticas y reutilizables.

El diseño sigue un patrón repetido en todas las vistas:

- **Navbar** fija en la parte superior, con rutas principales, accesos rápidos y notificaciones.
- **Breadcrumb** contextual para indicar la ruta actual.
- **Encabezado de sección** con el título y posibles botones de acción.
- **Cuerpo principal** con contenido adaptado al tipo de vista: formulario, listado, tabla, detalle, etc.

Este enfoque intenta mantener un diseño usable y accesible, haciendo que la aplicación sea intuitiva, predecible y agradable de usar.

Subtarea 3.2.1: Guía de estilos y personalización visual con Tailwind CSS

El sistema de diseño de **Tábula** se construyó completamente con **Tailwind CSS**, utilizando sus clases utilitarias para dar estilo a todos los componentes sin necesidad de escribir hojas de estilo personalizadas. A lo largo del desarrollo, se adoptaron una serie de convenciones visuales mínimas para mantener la coherencia visual en toda la aplicación.

Color principal y jerarquía visual

- El color base de la aplicación es **indigo**, utilizado principalmente en botones de acción y enlaces destacados:
 - Botones principales: `bg-indigo-600 hover:bg-indigo-500 text-white`
 - Enlaces activos o llamados a la acción: `text-indigo-600 hover:text-indigo-500`
- El fondo general de los formularios y secciones es **blanco**, sobre una base de `gray-100/gray-200` para los fondos generales.

Tipografía, tamaño y espaciado

- Se mantuvieron tamaños legibles:
 - Encabezados principales: `text-2xl font-bold`
 - Subtítulos: `text-lg font-semibold`
 - Texto en formularios y tablas: `text-sm text-gray-700`
- Los formularios utilizan `py-1 px-3` y `rounded` con sombras suaves (`shadow-sm`) para mejorar la accesibilidad visual y el foco.

Componentes visuales reutilizados

- **Botones primarios:** `bg-indigo-600`, con `hover` más claro y texto blanco.
- **Botones secundarios:** `bg-gray-200`, `bg-red-600` o `bg-green-600`, dependiendo de su función.
- **Inputs y selects:** se estandarizó el alto (`h-[36px]`) y el estilo de borde y foco.
- **Tarjetas visuales** (`div` con `bg-white`, `rounded`, `shadow`, `p-4`): utilizadas para resaltar secciones como gráficas, listados o formularios.



Inicia sesión

Correo electrónico

Contraseña

Entrar

¿No tienes cuenta? [Regístrate aquí](#)

Figura 14: Captura de página de login, Login.jsx, detalle de botón principal y campos input tipo “text”

Consistencia visual

Para mantener la consistencia, se evitaron estilos inline o clases personalizadas, apoyándose únicamente en la composición de utilidades Tailwind. Además, se crearon **componentes** como FiltrosContainer, SearchInput, SelectFilter, Pagination, etc., con diseño unificado.

Responsividad

Todas las vistas están adaptadas con **clases responsivas** (md:grid-cols-2, max-w-xl, w-full, etc.) que aseguran una presentación adecuada en pantallas pequeñas sin comprometer la usabilidad en escritorio, que es el enfoque principal de uso.

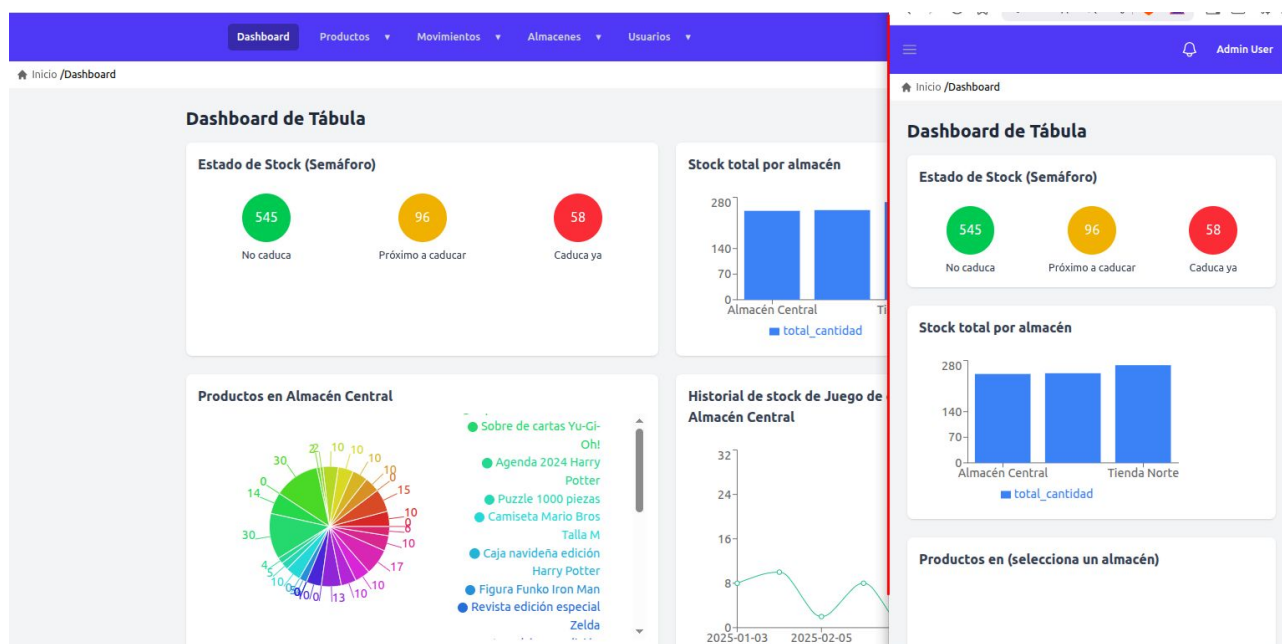


Figura 15: Captura de vistas Desktop (izquierda) y Móvil (derecha) del Dashboard principal.

Subtarea 3.2.2: Usabilidad y accesibilidad

Desde el inicio del diseño de la interfaz se buscó mantener un enfoque centrado en el usuario, asegurando que la aplicación fuera fácil de usar, intuitiva y accesible. Aunque no se siguieron todas las directrices de accesibilidad web (WCAG) de forma exhaustiva, sí se aplicaron varias buenas prácticas fundamentales que mejoran la experiencia para cualquier usuario.

Contraste y visibilidad

- Se utilizaron combinaciones de color con buen contraste entre texto y fondo (por ejemplo, blanco sobre indigo o gris oscuro sobre blanco).
- Los botones activos y enlaces utilizan colores destacados (text-indigo-600, hover:text-indigo-500) que permiten identificar claramente las acciones disponibles.

Indicadores visuales

- Todos los elementos interactivos (botones, inputs, selects) incluyen estados de hover y focus visibles mediante ring o cambio de color, lo que permite identificar el foco actual durante la navegación con teclado.

- Los formularios muestran mensajes de error debajo de los campos correspondientes, utilizando componentes como `<ErrorMessage />` con texto en rojo para mayor claridad.

Tamaños y espacios adecuados

- Los botones y campos de formulario mantienen un tamaño mínimo (h-[36px], py-2 px-4) suficiente para facilitar su clic o pulsación.
- Se empleó spacing y padding adecuados para evitar interfaces saturadas o confusas.

Navegación y estructura clara

- Todas las vistas siguen una estructura coherente: título, breadcrumb, cuerpo y acciones.
Esto facilita la orientación del usuario y reduce la carga cognitiva.
- Los formularios, tablas y listados se han diseñado con etiquetas (label) correctamente asociadas a sus inputs para favorecer el uso con lectores de pantalla y teclados.

Feedback y prevención de errores

- Se muestran alertas o mensajes claros tras acciones importantes (como crear, editar o eliminar), lo que aporta seguridad al usuario.
- Se previenen errores mediante validación básica en frontend (campos obligatorios, mínimos de caracteres, etc.), reduciendo intentos fallidos de envío.

Subtarea 3.2.3: Vistas tipo y componentes principales

Debido a la cantidad de páginas y contenido de esta aplicación, se ha optado por analizar las vistas más relevantes de la misma con el objetivo de no extender demasiado la longitud del presente documento. A continuación se pasa a comentar el diseño de algunas de las vistas y componentes más importantes.

Autenticación (Login y Registro)

Las primeras vistas con las que interactúa el usuario en la aplicación son las de **inicio de sesión** ([pages/Login.jsx](#)) y, opcionalmente, **registro** ([pages/Register.jsx](#)). Ambas comparten un diseño minimalista, centrado y enfocado en la acción principal.

Diseño común

- Se utiliza una tarjeta blanca (bg-white rounded shadow p-6) centrada verticalmente sobre fondo gris claro.
- Se emplean campos de entrada (input) con estilos estandarizados, validación básica (por ejemplo, email requerido) y mensajes de error visuales cuando es necesario.
- El botón de acción (Iniciar sesión, Registrarse) utiliza el diseño general del sistema: bg-indigo-600 text-white hover:bg-indigo-500.
- En la vista de login se incluye enlace rápido hacia “Registrarse” (si corresponde)

Accesibilidad y UX

- Los formularios cuentan con label correctamente asociados a cada campo.
- Se mantiene el enfoque del cursor en el primer campo al cargar la página.
- Al autenticarse correctamente, el usuario es redirigido al Dashboard.

Comportamiento

- Tras enviar el formulario, se comunica con el endpoint /auth/login.
- Si la autenticación es correcta, se guarda el access_token en memoria y se recibe el refresh_token en cookie.
- El sistema de rutas (AppRouter) gestiona automáticamente el acceso a vistas privadas, y evita que un usuario ya autenticado acceda a /login de nuevo.

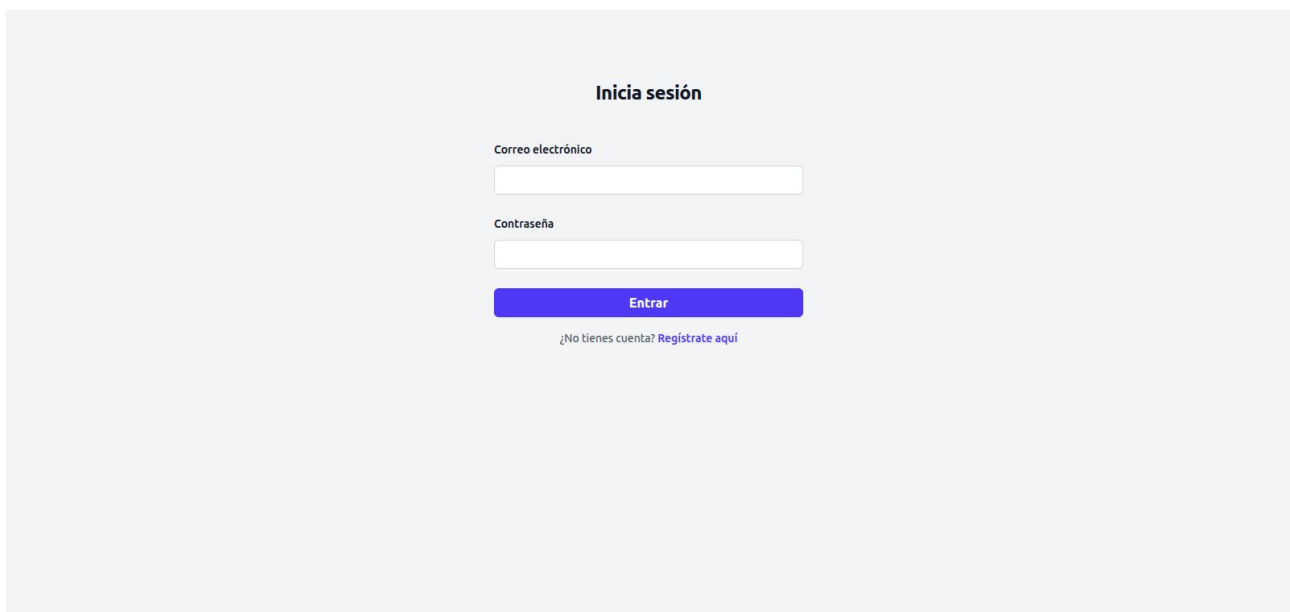


Figura 16: Captura de la vista de Login de la aplicación Tábula

Dashboard

El **Dashboard** actúa como la vista de inicio tras el login y proporciona una visión global del estado del inventario. Su diseño está enfocado en la **visualización de métricas clave** mediante tarjetas y gráficos, permitiendo al usuario obtener una lectura rápida de la situación actual.

Estructura y diseño

- **Encabezado con título** y breadcrumb contextual.
- **Tarjetas resumen** (div.bg-white.shadow.rounded.p-4) que muestran indicadores clave como:
 - Total de productos activos
 - Total de stock en almacenes
 - Cantidad de movimientos del último mes
- **Gráficas** integradas con la librería Recharts:
 - Gráfico circular con productos por categoría.
 - Gráfico de barras con volumen de stock por almacén.

- Gráfico temporal con evolución de movimientos en el último año.

Los componentes de gráfica se adaptan automáticamente al espacio disponible (ResponsiveContainer), y están integrados con la estética de la aplicación (bordes redondeados, tipografía consistente, colores contrastados).

Comportamiento

- Los datos se obtienen mediante llamadas a los endpoints /stock, /productos, /almacenes y /movimientos/last-year.
- Al hacer clic en una categoría o almacén en algunos gráficos, se redirige al listado correspondiente filtrado automáticamente (por ejemplo, productos de una categoría específica).

Experiencia de usuario

- El diseño se adapta visualmente a diferentes tamaños de pantalla, reorganizando las tarjetas y gráficos.
- Las gráficas están optimizadas para ser informativas pero no sobrecargadas, al posicionarse (hover) sobre las tarjetas que envuelven las gráficas, se muestra un mensaje informativo que indica al usuario cómo interactuar con ellas.

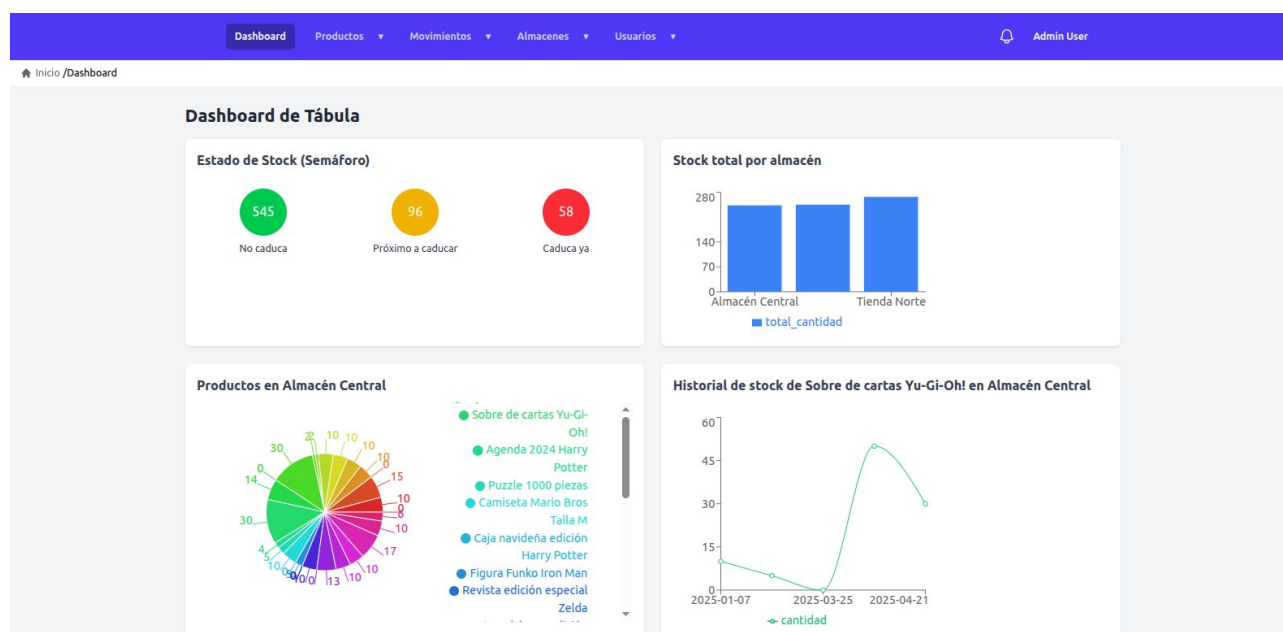


Figura 17: Captura de la vista del Dashboard principal, funciona como "Home" de la aplicación.

CRUD (Listado, Crear, Editar, Detalle)

Las vistas de gestión de entidades en la aplicación (productos, almacenes, usuarios, etc.) siguen un patrón común inspirado en las operaciones **CRUD**: crear, leer (listar y ver detalles), actualizar y eliminar. Este enfoque estructurado mejora la experiencia del usuario al ofrecer una interfaz coherente y predecible entre secciones.

Estructura base

Cada entidad gestionable (productos, almacenes, usuarios...) cuenta con algunas de las siguientes vistas, usaremos las vistas relacionadas con la gestión de productos para ilustrarlo.

1. Vista principal (home)

Actúa como punto de entrada visual de la sección. Puede incluir un resumen, gráficos o accesos rápidos (ej. /productos). Se accede desde el menú de navegación.

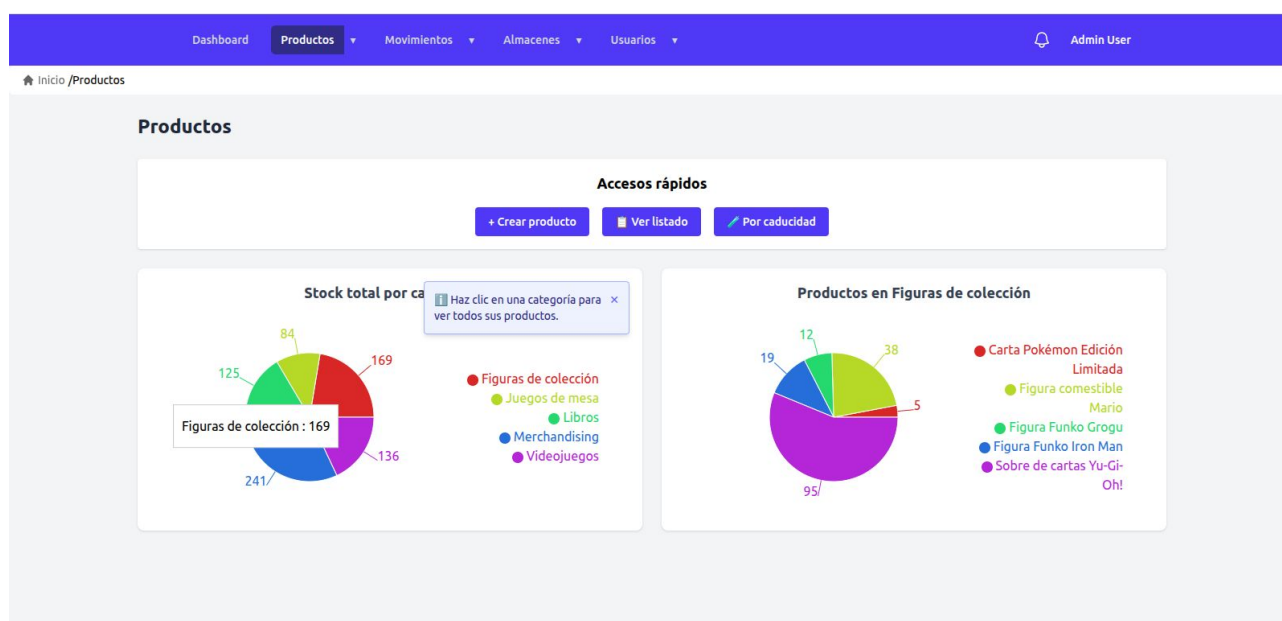


Figura 18: Captura de la página Productos.jsx

2. Listado paginado

- Tabla con filas dinámicas, filtros por campo (categoría, estado, búsqueda...).
- Acciones individuales: editar, eliminar, ver detalle.
- Acciones en bloque mediante checkboxes: activar/desactivar, exportar a CSV.
- Paginación, búsqueda instantánea y feedback visual.

The screenshot shows a web application interface for managing products. At the top is a navigation bar with links to Dashboard, Productos, Movimientos, Almacenes, and Usuarios, along with a user profile icon and 'Admin User'. Below the navigation bar is a breadcrumb trail: Inicio / Productos / Listado de productos. The main section is titled 'Listado de productos' and includes a '+ Crear producto' button. There are three filters: 'Buscar' (a text input with placeholder 'Buscar por nombre o SKU...'), 'Categoría' (a dropdown menu set to 'Todas las categorías'), and 'Estado' (a dropdown menu set to 'Todos'). An 'Exportar CSV' button is also present. The table below has columns for selection (checkbox), SKU, NOMBRE, CATEGORÍA, ACTIVO, and ACCIONES. It lists 10 products, each with a set of actions: 'Ver producto', 'Ver stock', 'Editar', and 'Eliminar'. At the bottom, there is a pagination bar showing 'Página 1 de 3' and 'Mostrando 1 a 10 de 30 resultados', along with 'Anterior' and 'Siguiente' buttons.

☐	SKU	NOMBRE	CATEGORÍA	ACTIVO	ACCIONES
☐	LIBRO01	1984 de George Orwell	Libros	Sí	Ver producto Ver stock Editar Eliminar
☐	AGENDA01	Agenda 2024 Harry Potter	Libros	Sí	Ver producto Ver stock Editar Eliminar
☐	AJEDR01	Ajedrez de Madera	Juegos de mesa	Sí	Ver producto Ver stock Editar Eliminar
☐	ENERGYPK	Bebida energética Pikachu	Merchandising	Sí	Ver producto Ver stock Editar Eliminar
☐	NAVHP01	Caja navideña edición Harry Potter	Merchandising	Sí	Ver producto Ver stock Editar Eliminar
☐	SURPRISE01	Caja sorpresa Pokémon	Merchandising	Sí	Ver producto Ver stock Editar Eliminar
☐	MERCH01	Camiseta Mario Bros Talla M	Merchandising	Sí	Ver producto Ver stock Editar Eliminar
☐	POKE01	Carta Pokémon Edición Limitada	Figuras de colección	Sí	Ver producto Ver stock Editar Eliminar
☐	JDM02	Catan	Juegos de mesa	Sí	Ver producto Ver stock Editar Eliminar
☐	CHICLE01	Chicle edición Zelda	Merchandising	Sí	Ver producto Ver stock Editar Eliminar

Figura 19: Captura de ListadoProductos.jsx

3. Formulario de creación

- Formulario con validación en tiempo real.
- Campos organizados y etiquetados (label, input, select, textarea).
- Botones Guardar y Cancelar.
- Componente adicional para crear categoría inline (CrearCategorialInline).

The screenshot shows the 'Nuevo producto' (New product) form. At the top, there is a navigation bar with links to Dashboard, Productos, Movimientos, Almacenes, and Usuarios, along with a user profile icon labeled 'Admin User'. Below the navigation bar, the breadcrumb trail reads 'Inicio / Productos / Nuevo producto'. The form itself is titled 'Nuevo producto' and contains the following fields: 'SKU' (text input), 'Nombre corto' (text input), 'Descripción' (text area), 'Categoría' (dropdown menu with 'Selecciona una categoría' as the placeholder), and a section for adding new categories with the text '¿No encuentras la categoría?' and a 'Nueva categoría' input field. At the bottom of the form, there are three buttons: 'Guardar producto' (blue), 'Cancelar' (gray), and 'Añadir' (blue).

Figura 20: Vista de la página `NuevoProducto.jsx`, aquí se puede añadir un nuevo producto al inventario.

4. Formulario de edición

Similar al de creación, pero con campos precargados y lógica condicional para proteger ciertos valores según el rol del usuario.

The screenshot shows the 'Editar producto' (Edit product) form. The navigation bar and breadcrumb trail are the same as in Figure 20. The form is titled 'Editar producto' and contains the following fields: 'SKU' (text input with 'LIBRO01' pre-filled), 'Nombre corto' (text input with '1984 de George Orwell' pre-filled), 'Descripción' (text area with 'Novela distópica' pre-filled), and 'Categoría' (dropdown menu with 'Libros' pre-filled). Below these fields, there is a section for adding new categories with the text '¿No encuentras la categoría?' and a 'Nueva categoría' input field. At the bottom of the form, there are three buttons: 'Guardar cambios' (blue), 'Cancelar' (gray), and 'Eliminar producto' (red). There is also a checkbox labeled 'Activo' which is checked.

Figura 21: Captura de la vista `EditarProducto.jsx`

5. Vista de detalle

- Tarjeta con todos los campos clave del recurso.
- Botón de acción contextual: volver atrás o editar (según permisos).
- Se utiliza para consulta rápida y navegación contextual desde otras vistas.

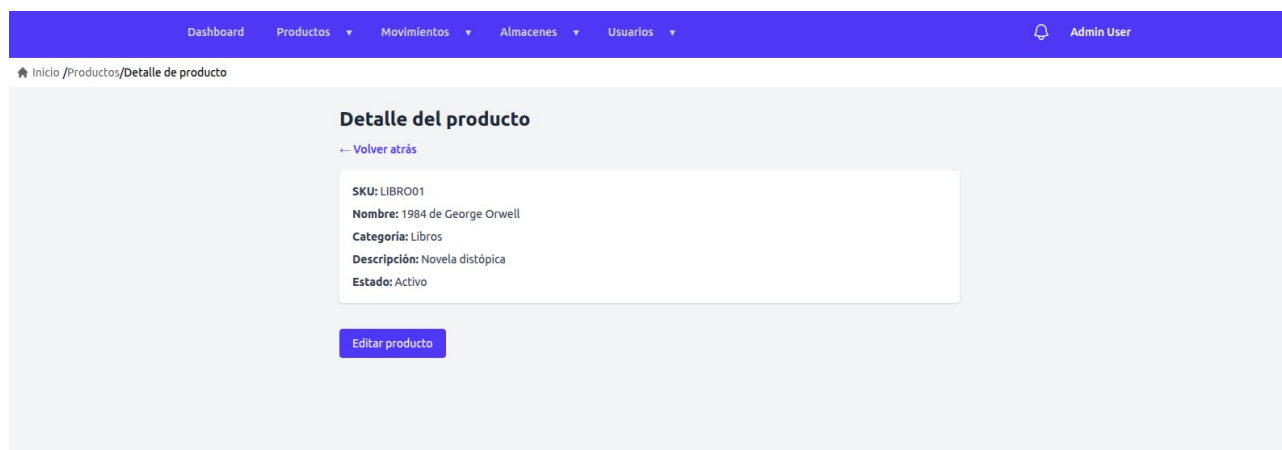


Figura 22: Captura de la vista DetalleProducto.jsx

Estilo visual y coherencia

- Todas las vistas CRUD comparten componentes visuales comunes: Navbar, Breadcrumb, botones, formularios, tablas y filtros.
- Se utilizan tarjetas (bg-white shadow rounded) para encapsular formularios y secciones.
- Los inputs siguen el mismo estilo (h-[36px], focus:ring-indigo-500, etc.) y la validación se gestiona de forma centralizada (ErrorMessage).
- En los listados, las filas alternan colores para mayor legibilidad y las tablas responden a tamaños de pantalla.

Usabilidad

- La experiencia está optimizada para usuarios frecuentes: acciones rápidas, navegación clara, confirmaciones por alerta y mensajes de error/éxito inmediatos.
- El usuario puede realizar operaciones en bloque y filtrar sin recargar la vista.
- Los formularios restablecen el estado tras enviar correctamente y redirigen automáticamente al listado.

Stock e históricos

Las vistas relacionadas con el **stock** y los **históricos de movimientos** están diseñadas para proporcionar una visión detallada del inventario actual y su evolución a lo largo del tiempo. Estas vistas combinan tablas y gráficas, ofreciendo tanto información operativa como contexto visual.

Stock por producto (src/pages/StockPorProducto.jsx)

- Muestra el stock total de un producto distribuido por almacén.
- Utiliza el componente **src/components/TablaResumenStock** para presentar:
 - Nombre del almacén
 - Código
 - Cantidad total disponible
- Incluye acciones para ver el historial de ese producto en un almacén concreto o su historial global.
- Accesible desde el listado de productos, vía botón “Ver stock”.

Dashboard Productos Movimientos Almacenes Usuarios Admin User

Inicio / Productos / Detalle de producto / Stock del producto

Stock del producto LIBRO01

[← Volver atrás](#)

ALMACÉN	CÓDIGO ALMACÉN	CANTIDAD TOTAL	ACCIONES
Almacén Central	1	0	Ver historial en almacén
Tienda Norte	2	9	Ver historial en almacén
Tienda Metropolitana	3	4	Ver historial en almacén
Total general		13	Ver historial total

Página 1 de 1 — Mostrando 1 a 3 de 3 resultados

[Anterior](#) [Siguiente](#)

Figura 23: Captura de stock por producto

Stock por almacén ([StockPorAlmacen.jsx](#))

- Muestra todos los productos almacenados en un almacén específico.
- Usa [TablaResumenStockAlmacen](#), con columnas:
 - Producto, SKU, cantidad y botón para ver historial.
- Permite navegar a:
 - Historial total del almacén
 - Historial de cada producto dentro del almacén
- Incluye paginación y recuperación del nombre del almacén.

Stock de un producto en un almacén ([StockProductoPorAlmacen.jsx](#))

- Vista detallada del stock de **un único producto en un almacén** concreto.
- Incluye:
 - Tabla con lotes, cantidades y fecha de caducidad
(`<td>{item.fecha_cad}</td>`)
 - Gráfico de línea con evolución de stock (usando recharts)
- Datos obtenidos del endpoint `/stock/almacen/{codigo}/producto/{codigo}`.
- Incluye navegación de retorno clara.

Inicio / Stock por almacén / Histórico del producto en almacén



Figura 24: Captura de la vista de stock de un producto en un almacén concreto

Historial por producto ([HistorialProducto.jsx](#))

- Vista que muestra todos los movimientos (entradas/salidas) de un producto a lo largo del tiempo.
- Se presenta:
 - Gráfico de evolución temporal (acumulado de stock)
 - Tabla de movimientos con datos como tipo, cantidad, lote, almacén, usuario.
- Incluye paginación y back automático.

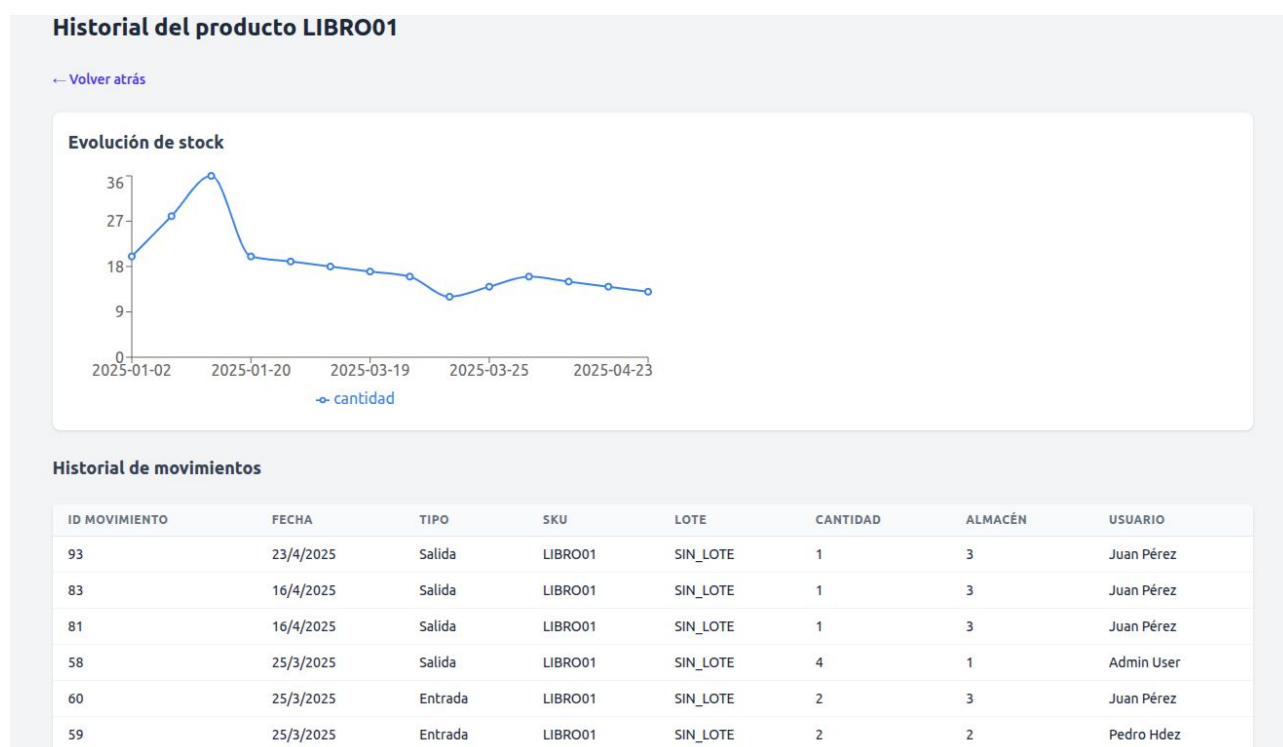


Figura 25: Captura de la vista del histórico de movimientos totales de un producto

Historial por almacén ([HistorialStockAlmacen.jsx](#))

- Similar a la anterior, pero centrada en un almacén concreto.
- Gráfico muestra el stock acumulado total del almacén a lo largo del tiempo.
- La tabla detalla todos los movimientos asociados, por producto.

Historial de producto en almacén ([HistorialProductoPorAlmacen.jsx](#))

- Vista combinada que detalla los movimientos de un producto específico **en un almacén concreto**.

- Permite analizar en profundidad el comportamiento del stock cruzando producto y ubicación.
- Estructura similar a las otras vistas de historial (gráfico + tabla).

Productos por caducidad ([StockProductoCaducidad.jsx](#))

- Vista especializada que muestra **productos próximos a caducar** en un rango de fechas configurable.
- Presenta:
 - **Gráfico de barras** que resume cuántos productos caducan en cada tramo temporal.
 - **Tabla** ([TablaStockProductoCaducidad](#)) que muestra sku, lote, fecha caducidad, etc.
- Es accesible desde el dashboard y desde el menú desplegable.

Esta vista permite detectar con antelación productos perecederos, facilitando decisiones como reorganización, redistribución o retirada de stock.

Navbar (navegación principal)

El componente Navbar constituye la cabecera fija de la aplicación y cumple varias funciones críticas:

- **Navegación principal:** contiene enlaces a las secciones principales del sistema (Dashboard, Productos, Movimientos, Almacenes, Usuarios), con submenús accesibles según el rol del usuario (admin o usuario).
- **Menú responsive:** adapta su diseño automáticamente en dispositivos móviles mediante el componente Disclosure de Headless UI.
- **Notificaciones en tiempo real:** integra una **campana de notificaciones** que recibe nuevos mensajes vía WebSocket cuando se crea un nuevo movimiento. Estas notificaciones se almacenan en localStorage y se muestran en un desplegable.
- **Menú de usuario:** incluye accesos al perfil y la opción de cerrar sesión.

El diseño se basa en Tailwind CSS y Headless UI, usando iconos de Heroicons. Se mantiene coherencia visual con el resto del sistema (color indigo, hover en blanco, texto claro).

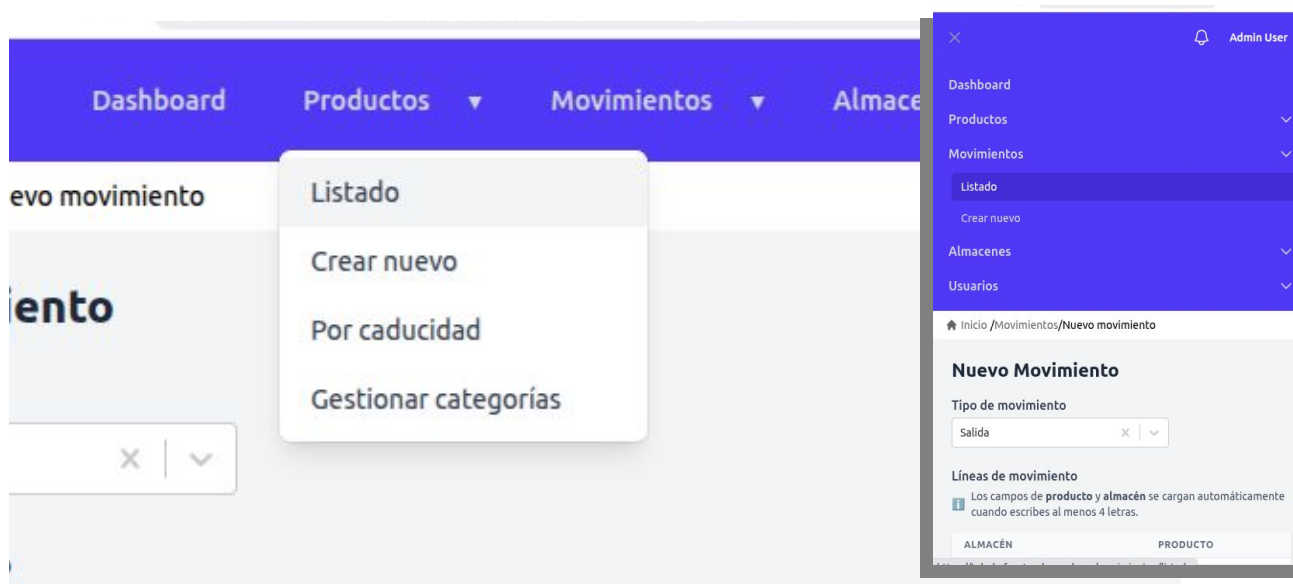


Figura 26: Capturas de la barra de navegación, versión desplegable de menú desktop (izquierda), versión responsive (derecha)

Crear movimiento (CrearMovimiento.jsx)

Esta vista permite registrar **movimientos de stock**, tanto de entrada como de salida, mediante un formulario dinámico que permite añadir múltiples líneas de producto.

Diseño y comportamiento

- Encabezado con título y breadcrumb.
- Selector del tipo de movimiento (entrada/salida).
- Tabla editable donde cada fila representa una línea del movimiento:
 - Almacén (select asíncrono)
 - Producto (select asíncrono)
 - Lote (input o select según el tipo de movimiento)
 - Fecha de caducidad

- Cantidad
- Botón para eliminar línea
- Botón para añadir más líneas.
- Validaciones de campos con mensajes de error (`src/components/ErrorMessage`).
- Carga automática de lotes disponibles cuando se selecciona producto y almacén en movimientos de salida.
- Al guardar, se envía una petición al backend y se redirige al listado de movimientos.

El formulario mantiene la estética general de la aplicación: tarjetas blancas, inputs estilizados, botones indigo, espaciado uniforme, y diseño responsive.

Nuevo Movimiento

Tipo de movimiento

Salida

Líneas de movimiento

Los campos de **producto** y **almacén** se cargan automáticamente cuando escribes al menos 4 letras.

ALMACÉN	PRODUCTO	LOTE	FECHA CADUCIDAD	CANTIDAD	ACCIONES
Almacén Central	Snack temático One Piece	OP_201234	01/06/2025	10	X

+ Añadir línea

Guardar movimiento

Figura 27: Captura de la vista de `CrearMovimiento.jsx`

Subtarea 3.3: Implementación de la lógica de autenticación y rutas

Subtarea 3.3.1: Estructura base y montaje de la aplicación

La aplicación React está estructurada de forma jerárquica, envolviendo el componente principal `<App />` en dos capas fundamentales, mostramos a continuación el detalle del contenido del archivo `src/main.jsx`:

```
10 ReactDOM.createRoot(document.getElementById("root")).render(  
11   <React.StrictMode>  
12     <BrowserRouter>  
13       <AuthProvider>  
14         <App />  
15       </AuthProvider>  
16     </BrowserRouter>  
17   </React.StrictMode>  
18 );  
19  
20
```

Figura 28: Captura detalle del contenido del archivo `main.jsx`, Vite usa este archivo como punto de entrada. Se selecciona el elemento "root" de `index.html`, elemento donde se va a renderizar toda la aplicación.

Esto permite:

- **<BrowserRouter>**: gestionar el sistema de rutas de la aplicación usando `react-router-dom`, haciendo que cada URL renderice un componente diferente.
- **<AuthProvider>**: crear un **contexto global de autenticación**, accesible desde cualquier componente de la aplicación mediante el hook `useAuth()`.
- **<React.StrictMode>**: Modo estricto que ayuda a detectar errores potenciales en desarrollo. Solo afecta al entorno de desarrollo, no es una capa fundamental.

El componente `<App />` (`src/App.jsx`) actúa como contenedor general. En su interior se encuentra `<AppRouter />`, que define todas las rutas disponibles mediante `<Routes>` y `<Route>`.

```

src > App.jsx > ...
1  import AppRouter from "../router/AppRouter";
2
3  /*
4   Dentro de este contenedor se renderiza <AppRouter />, lo que significa que TODO
5   lo que se renderiza en la app depende de las rutas definidas ahí.
6   */
7
8  function App() {
9    return (
10     <div className="min-h-screen bg-gray-100">
11       <AppRouter />
12     </div>
13   );
14 }
15
16 export default App;
17

```

Figura 29: Captura del archivo App.jsx, contenedor general de la aplicación

Subtarea 3.3.2: Contexto de autenticación (src/context/AuthProvider)

React Context permite crear un "estado global" compartido entre componentes sin necesidad de **prop drilling**^[9]. En este proyecto se ha creado el archivo [AuthProvider.jsx](#), que implementa este contexto, para almacenar y manejar de forma global todo lo relacionado con la **autenticación del usuario**. Esto incluye el estado del usuario, los tokens, el login/logout, y el refresco automático del access_token.

Al envolver toda la app con este componente, se consigue que cualquier otro componente pueda acceder a esta información sin tener que pasarla por props.

Funcionamiento general:

- **createContext()**: crea el contexto AuthContext.
- **AuthProvider**: es el componente que contiene toda la lógica relacionada con la autenticación y que proporciona el contexto.
- **Estados gestionados**:
 - **accessToken**: token JWT de acceso.

- user: objeto con datos del usuario autenticado (nombre, email, rol...).
- isLoading: controla el estado de carga inicial.
- isLoggingOut: controla si se está ejecutando un logout.
- **Funciones clave:**
 - login(): inicia sesión, guarda el token, programa su refresco y obtiene el perfil.
 - logout(): elimina la cookie de sesión y limpia el estado local.
 - refreshAccessToken(): renueva el token usando una cookie HttpOnly.
 - scheduleTokenRefresh(): programa un setTimeout para renovar el token antes de que caduque.

Uso del contexto:

Para acceder a esta información en cualquier parte de la app, se utiliza el **hook personalizado useAuth()**, definido en **src/context/[useAuth.js](#)**:

Subtarea 3.3.3: Sistema de rutas protegidas y navegación condicional

Tábula utiliza **react-router-dom** para definir las distintas rutas de la aplicación, y extiende su funcionalidad mediante componentes personalizados que controlan el acceso a ciertas vistas dependiendo del **estado de autenticación** y el **rol del usuario**.

src/router/[AppRouter.jsx](#): definición central de rutas

Este archivo contiene el mapa completo de rutas de la aplicación:

Claves del diseño:

- Se usa **<Route element={...}>** para agrupar rutas que comparten lógica condicional:
 - PublicRoute: vistas accesibles solo si no estás logueado.
 - PrivateRoute: requiere estar autenticado.
 - AdminRoute: requiere además tener rol admin.

```

35  /*
36  Routes, Route: vienen de react-router-dom, y permiten declarar las rutas (como en un switch-case).
37  Navigate: sirve para redirigir de una ruta a otra.
38  */
39
40  export default function AppRouter() {
41    return (
42      //Dentro de <Routes>, se colocan los <Route> que representan las distintas páginas.
43      <Routes>
44        <Route path="/" element={<Navigate to="/login" />} />
45
46        <Route element={<PublicRoute />>
47          <Route path="/login" element={<Login />} />
48          <Route path="/register" element={<Register />} />
49        </Route>
50
51        <Route element={<AdminRoute />>
52          <Route path="/usuarios" element={<Usuarios />} />
53          <Route path="/usuarios/nuevo" element={<NuevoUsuario />} />
54          <Route path="/usuarios/:id/editar" element={<EditarUsuario />} />
55          <Route path="/productos/nuevo" element={<NuevoProducto />} />
56          <Route path="/productos/:id/editar" element={<EditarProducto />} />
57          <Route path="/almacenes/nuevo" element={<NuevoAlmacen />} />
58          <Route path="/almacenes/:codigo/editar" element={<EditarAlmacen />} />
59          <Route path="/categorias" element={<Categorias />} />
60        </Route>
61
62        <Route element={<PrivateRoute />>
63          <Route path="/dashboard" element={<Dashboard />} />
64          <Route path="/productos" element={<Productos />} />
65          <Route path="/movimientos" element={<Movimientos />} />
66          <Route path="/almacenes" element={<Almacenes />} />
67          <Route path="/perfil" element={<Perfil />} />
68          <Route path="/caducidad" element={<StockProductoCaducidad />} />
69          <Route path="/movimientos/nuevo" element={<CrearMovimiento />} />
70          <Route path="/movimientos/listado" element={<ListadoMovimientos />} />
71          <Route path="/movimientos/:id" element={<DetalleMovimiento />} />
72          <Route path="/productos/listado" element={<ListadoProductos />} />

```

Figura 30: Captura con detalle del contenido de AppRouter.jsx

PublicRoute

Este componente restringe el acceso a rutas públicas como el login o el registro cuando ya existe una sesión activa. Si el usuario está autenticado, lo redirige directamente al dashboard.

Uso típico: evitar que un usuario ya logueado vuelva a acceder al formulario de login.

```
src > router > PublicRoute.jsx > ...
1 import { Navigate, Outlet } from "react-router-dom";
2 import { useAuth } from "../context/useAuth";
3
4 export default function PublicRoute() {
5   const { isAuthenticated } = useAuth();
6
7   return isAuthenticated ? <Navigate to="/dashboard" replace /> : <Outlet />;
8 }
9
```

Figura 31: Captura de *PublicRoute.jsx*

PrivateRoute

Encargado de proteger las vistas privadas del sistema. Solo permite el acceso si el usuario está autenticado. En caso contrario, redirige automáticamente al login. Este componente es esencial para la seguridad de la aplicación.

Uso típico: todas las rutas que requieren sesión activa, como el dashboard o la gestión de productos.

AdminRoute

Es una extensión del componente privado, que además verifica que el usuario tenga rol de administrador (rol === "admin"). Si no se cumplen los requisitos, redirige a una ruta de acceso no autorizado o simplemente restringe la visualización.

Uso típico: rutas como gestión de usuarios, categorías o creación/edición de productos y almacenes.

Subtarea 3.4: Implementación de vistas dinámicas y componentes funcionales con React

Los archivos correspondientes se encuentran en [tabula-frontend/src/components](#) y [tabula-frontend/src/pages](#). Dado el elevado número de vistas y componentes implementados, en esta sección nos centraremos en la página del *Dashboard* y en dos componentes clave: *Navbar* y *Breadcrumb*. Este enfoque permite ilustrar de forma representativa la lógica general de funcionamiento del frontend sin necesidad de detallar todos los archivos.

Subtarea 3.4.1: Vistas de la aplicación, lógica reactiva y uso de hooks

La vista del **Dashboard** de Tábula es un buen ejemplo para ilustrar cómo se han manejado las **peticiones a la API**, **gestión de estado local**, **efectos secundarios** y **representación visual de datos** mediante componentes gráficos reutilizables.

A nivel técnico, en [Dashboard.jsx](#) se utilizan principalmente los siguientes elementos clave de React:

- **useState** para almacenar los datos recibidos del backend.
- **useEffect** para lanzar las peticiones al montarse el componente.
- **Funciones importadas desde api/** para centralizar la lógica de consulta HTTP (por ejemplo, `getStockTotal`, `getStockPorProducto`, etc.).
- **Gráficos renderizados con Recharts** (gráfico de barras, gráfico tipo tarta, etc.).
- **Componentes auxiliares reutilizables**, como el *Breadcrumb* o el *Navbar*.

La estructura de la vista se puede dividir en tres bloques funcionales:

1. Inicialización del estado

Se definen variables de estado para guardar los datos de stock total por almacén, stock por producto, productos próximos a caducar, etc.


```

28 export default function Dashboard() {
29   const { accessToken } = useAuth();
30   const navigate = useNavigate();
31   // Estados para los datos de los 4 gráficos
32   const [semaforo, setSemaforo] = useState(null);
33   const [almacenes, setAlmacenes] = useState([]);
34   const [productosAlmacen, setProductosAlmacen] = useState([]);
35   const [almacenSeleccionado, setAlmacenSeleccionado] = useState(null);
36   const [nombreAlmacenSeleccionado, setNombreAlmacenSeleccionado] =
37     useState(null);
38   const [historialProducto, setHistorialProducto] = useState([]);
39   const [nombreProductoSeleccionado, setNombreProductoSeleccionado] =
40     useState(null);
41 }

```

Figura 32: Captura VS Code de detalle en Dashboard.jsx, declaración e inicialización de los estados con useState

2. Lanzamiento de peticiones con useEffect

Dentro de un useEffect, se agrupan todas las funciones asincrónicas que consultan la API al cargar la página:

```

42 const fetchData = async () => {
43   try {
44     const [dataSemaforo, dataAlmacenes] = await Promise.all([
45       getSemaforo(accessToken),
46       getDetalleAlmacenes(accessToken),
47     ]);
48     setSemaforo(dataSemaforo);
49     setAlmacenes(dataAlmacenes);
50   } catch (error) {
51     console.error("Error al obtener datos del dashboard:", error.message);
52   }
53 };
54
55 // Cargar datos iniciales
56 useEffect(() => {
57   if (!accessToken) return;
58   fetchData();
59 }, [accessToken]);

```

Figura 33: Captura de detalle Dashboard.jsx, se muestra el useEffect que maneja las peticiones fetch a los endpoints de stock (centralizadas en /src/api/stock.js)

3. Renderizado condicional + componentes gráficos

Finalmente, los datos obtenidos se muestran usando componentes gráficos de Recharts, y se incluyen condicionales para mostrar mensajes si no hay datos.


```

232      {/* Gráfico 4: Línea historial de stock */}
233      <div className="bg-white rounded-lg shadow p-4">
234        <h2 className="text-lg font-bold text-gray-700 mb-6">
235          Historial de stock de{ " "}
236          {nombreProductoSeleccionado}
237          ? `${nombreProductoSeleccionado} en ${nombreAlmacenSeleccionado}`
238          : "(selecciona un producto)"
239        </h2>
240        <div className="overflow-x-auto">
241          <div className="min-w-[400px]">
242            <LineChart width={350} height={300} data={historialProducto}>
243              <XAxis dataKey="fecha" />
244              <YAxis />
245              <Tooltip />
246              <Legend />
247              <Line type="monotone" dataKey="cantidad" stroke="#10b981" />
248            </LineChart>
249          </div>
250        </div>
251      </div>

```

Figura 34: Captura de detalle Dashboard.jsx, renderizado condicional de información

Subtarea 3.4.2: Componentes de la aplicación

Navbar: navegación principal y gestión de notificaciones

El componente Navbar centraliza la navegación de la aplicación. Se encuentra ubicado en [src/components/Navbar.jsx](#) y su funcionalidad se organiza en torno a tres bloques principales: (1) definición dinámica del menú, (2) integración con WebSockets para mostrar notificaciones en tiempo real, y (3) adaptabilidad visual en función del tamaño de pantalla y del rol del usuario.

1. Menú de navegación condicional

El array navigation se construye dinámicamente en función del rol del usuario (user?.role). Este objeto contiene tanto rutas principales como submenús agrupados bajo secciones como "Productos", "Movimientos", "Almacenes" o "Usuarios". Por ejemplo, si el usuario tiene rol admin, se añade automáticamente la sección "Usuarios" al menú. Además, se filtran los submenús para que solo se muestren opciones permitidas por el rol actual:

```

47   const navigation = [
48     { name: "Dashboard", href: "/dashboard" },
49     {
50       name: "Productos",
51       href: "/productos",
52       submenu: [
53         { name: "Listado", href: "/productos/listado" },
54         user?.role === "admin" && {
55           name: "Crear nuevo",
56           href: "/productos/nuevo",
57         },
58         { name: "Por caducidad", href: "/caducidad?desde=0&hasta=6" },
59         { name: "Gestionar categorías", href: "/categorias" },
60       ].filter(Boolean),
61     },
62     {
63       name: "Movimientos",
64       href: "/movimientos"

```

Figura 35: Captura detalle del componente Navbar.jsx, mapa del menú de navegación

2. Notificaciones en tiempo real

Navbar también se encarga de mostrar notificaciones recibidas vía WebSocket. Se utiliza un hook personalizado useWebSocket que recibe un callback para actualizar el estado local. Veremos más sobre esto en la sección correspondiente, [Tarea 4: Websocket y Despliegue](#).

Estas notificaciones se guardan localmente y pueden eliminarse individualmente desde el dropdown visual que se despliega al hacer clic sobre el icono de campana.

3. Diseño responsive y menú de usuario

El componente utiliza Disclosure y Menu de @headlessui/react para manejar el menú tanto en versión escritorio como móvil. En la versión móvil, se renderiza un panel colapsable con todos los enlaces. En escritorio, se muestra una barra horizontal con acceso directo y submenús. El nombre del usuario autenticado aparece en la parte superior derecha junto con un botón para cerrar sesión (logout).

Breadcrumb

El componente [Breadcrumb](#) permite mostrar una ruta de navegación contextual en la parte superior de cada vista. Su lógica se basa en interpretar dinámicamente la URL

actual y traducirla a nombres legibles, mostrando así una jerarquía comprensible para el usuario.

La estructura del componente puede dividirse en tres partes principales:

1. Mapeo de rutas a nombres visibles

Se define un objeto **breadcrumbNameMap** donde cada ruta de la aplicación se asocia a un nombre amigable. Se incluyen también rutas con parámetros dinámicos (como `:id` o `:codigo_almacen`), lo que permite resolver rutas individuales como `productos/123/editar`.

Además, se incluye un objeto **breadcrumbOverrides** para manejar excepciones: en rutas donde la estructura URL no refleja bien la jerarquía conceptual, se fuerza una estructura manual (por ejemplo, para mostrar “Almacenes > Listado de almacenes > Stock por almacén”).

2. Generación dinámica de los items

Al renderizar, el componente analiza **location.pathname** y genera los items del breadcrumb de dos formas:

- Si existe una coincidencia en los overrides, usa directamente esa estructura.
- Si no, construye los segmentos dinámicamente a partir del pathname, utilizando el objeto **breadcrumbNameMap** para traducir cada segmento acumulado.
- Utiliza las funciones auxiliares **matchOverride()** y **getBreadcrumbLabel()**

El resultado es una lista de objetos con `path`, `label` e `isLast`, que se utilizan para construir visualmente el breadcrumb.

```

88 export default function Breadcrumb() {
89   const location = useLocation();
90   const pathname = location.pathname;
91
92   // Si la ruta tiene override, lo usamos
93   const override = matchOverride(pathname);
94   const items = [];
95
96   if (override) {
97     // Si hay override, lo usamos tal cual
98     override.forEach((item, index) => {
99       items.push({
100         path: item.path,
101         label: item.label,
102         isLast: index === override.length - 1,
103       });
104     });
105   } else {
106     // Si no hay override, construimos el breadcrumb desde los segmentos reales de la ruta
107     const segments = pathname.split("/").filter(Boolean);
108     let pathAccumulator = "";
109
110     segments.forEach((segment, index) => {
111       pathAccumulator += `/${segment}`;
112       const label = getBreadcrumbLabel(pathAccumulator);
113       if (label) {
114         items.push({
115           path: pathAccumulator,
116           label,
117           isLast: index === segments.length - 1,
118         });
119       }
120     });
121   }

```

Figura 36: Captura detalle de *Breadcrumb.jsx*, generación dinámica de los items

3. Renderizado

El componente renderiza siempre como primer elemento un enlace a "Inicio" (el dashboard). Luego, recorre la lista de items:

- Si un ítem es el último, se muestra como texto plano.
- Si no, se renderiza como un enlace navegable (<Link>).

De esta forma, el breadcrumb permite al usuario orientarse y navegar fácilmente hacia niveles superiores.

```

123  return (
124    <Breadcrumbs className="bg-white px-4 py-2">
125      { /* Primer ítem: siempre es el inicio */ }
126      <Link to="/dashboard" className="opacity-60 flex items-center gap-1 mr-1">
127        <svg
128          xmlns="http://www.w3.org/2000/svg"
129          className="h-4 w-4"
130          viewBox="0 0 20 20"
131          fill="currentColor"
132        >
133          <path d="M10.707 2.293a1 1 0 00-1.414 0l-7 7a1 1 0 001.414 1.414L4 10.414"
134        </svg>
135        Inicio
136      </Link>
137
138      { /* Resto de items: enlaces o texto plano según si es el último */ }
139      { items.map(({ path, label, isLast }) =>
140        isLast || !path ? (
141          <span key={label}>{label}</span>
142        ) : (
143          <Link key={path} to={path} className="opacity-60">
144            {label}
145          </Link>
146        )
147      ) }
148    </Breadcrumbs>
149  );
150 }
151

```

Figura 37: Captura detalle del renderizado del componente Breadcrumb

El usuario verá algo como:



Figura 38: Captura detalle de breadcrumb dinámico

Duración

Tarea/Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 3: Desarrollo del Frontend (React y Tailwind)		17/03/2025	16/04/2025
Subtarea 3.1: Formación y configuración del entorno frontend		17/03/2025	23/03/2025
3.1.1: Formación en React y fundamentos SPA	15	17/03/2025	22/03/2025
3.1.2: Configuración del entorno de desarrollo	1	22/03/2025	23/03/2025
Subtarea 3.2: Diseño general de la interfaz de usuario (UI)		24/03/2025	27/03/2025
3.2.1: Guía de estilos y personalización visual con Tailwind CSS	8	24/03/2025	25/03/2025
3.2.2: Usabilidad y accesibilidad	4	25/03/2025	26/03/2025
3.2.3: Vistas tipo y componentes principales (diseño y comportamiento)	10	26/03/2025	27/03/2025
Subtarea 3.3: Implementación de la lógica de autenticación y rutas		28/03/2025	03/04/2025
3.3.1: Estructura base y montaje de la aplicación	3	28/03/2025	28/03/2025
3.3.2: Contexto de autenticación (AuthProvider)	10	29/03/2025	01/04/2025
3.3.3: Sistema de rutas protegidas y navegación condicional	7	02/04/2025	03/04/2025
Subtarea 3.4: Implementación de vistas dinámicas y componentes funcionales		04/04/2025	20/04/2025
└ 3.4.1: Vistas de la aplicación, lógica reactiva y uso de hooks	16	04/04/2025	12/04/2025
└ 3.4.2: Componentes de la aplicación (Navbar, Breadcrumb, etc.)	16	13/04/2025	20/04/2025
TOTAL	90		

Tarea 4: WebSocket y Despliegue

Esta tarea consiste en la integración de WebSockets para añadir capacidades de actualización en tiempo real a la aplicación, así como los pasos iniciales para su despliegue en un entorno accesible públicamente. Se plantearon inicialmente varias funcionalidades en tiempo real, pero se decidió centrarse en una implementación mínima viable y funcional para poder llegar al deadline del proyecto: **actualizar la campana de notificaciones del Navbar cada vez que se registra un nuevo movimiento de stock**. Esta funcionalidad permite alertar al usuario de cambios recientes sin necesidad de recargar la página.

Subtarea 4.1: Desarrollo de la funcionalidad de WebSocket

Se decidió implementar **un único canal WebSocket** en la ruta `/ws/movimientos`, reutilizado para notificaciones generales relacionadas con movimientos de stock. Esta ruta está disponible en el backend y se conecta desde el frontend React a través de un custom hook.

A continuación se analiza cada uno de los componentes implicados en sus respectivas subtareas:

Subtarea 4.1.1: Backend

1. routers/[websocket.py](#)

Este archivo define un **ConnectionManager** que gestiona las conexiones activas y permite hacer **broadcasts** a todos los clientes conectados:


```

7 class ConnectionManager:
8     """Esta clase guarda todas las conexiones activas en una lista.
9     Cada vez que alguien se conecta al WebSocket, se añade a esta lista."""
10
11     def __init__(self):
12         self.active_connections: List[WebSocket] = []
13
14     async def connect(self, websocket: WebSocket):
15         await websocket.accept() # .accept() es obligatorio para establecer la conexión con el cliente.
16         self.active_connections.append(
17             websocket
18         ) # Después, lo guardamos en la lista de conexiones activas.
19
20     def disconnect(self, websocket: WebSocket):
21         self.active_connections.remove(
22             websocket
23         ) # Si el cliente se desconecta (o se cae), lo quitamos de la lista.
24
25     async def broadcast(self, message: str):
26         """Este método envía un mensaje de texto a todos los clientes conectados."""
27         for connection in self.active_connections:
28             await connection.send_text(message)
29

```

Figura 39: Captura de la clase ConnectionManager para gestionar las conexiones activas del websocket

La ruta `@router.websocket("/ws/movimientos")` mantiene viva la conexión:

```

31 # Instanciamos para poder usarla en cualquier parte del código
32 manager = ConnectionManager()
33
34
35 @router.websocket("/ws/movimientos")
36 async def websocket_endpoint(websocket: WebSocket):
37     # Llamamos a manager.connect() para aceptar la conexión y guardarla.
38     await manager.connect(websocket)
39
40     try:
41         # Mantenemos la conexión activa y viva con un bucle infinito.
42         while True:
43             await websocket.receive_text()
44     except WebSocketDisconnect:
45         # Cuando el cliente se desconecta, lo removemos para no dejar conexiones zombis.
46         manager.disconnect(websocket)
47

```

2. routers/[movements.py](#)

En la función `create_movement`, tras registrar un movimiento, se emite un mensaje a todos los clientes conectados vía WebSocket:


```

367 # Enviar mensaje a todos los clientes WebSocket conectados
368 try:
369     mensaje = (
370         f"Nuevo movimiento registrado: {new_movement.id_mov} ({new_movement.tipo})"
371     )
372
373     # Función asíncrona que realizará el broadcast del mensaje
374     async def emitir_websocket_mensaje(mensaje: str):
375         await manager.broadcast(mensaje)
376
377     """
378     Se utiliza AnyIO para emitir un mensaje WebSocket de manera asíncrona desde una ruta sincrónica,
379     garantizando compatibilidad con el event loop de FastAPI."
380     """
381     anyio.from_thread.run(emitir_websocket_mensaje, mensaje)
382
383 except Exception as e:
384     print("Error al emitir WebSocket:", str(e))
385

```

Figura 40: Captura de movements.py, detalle de la creación del mensaje de notificación que se transmitirá vía websocket en el endpoint /movimientos, create_movement(), justo después de crear un nuevo movimiento.

Se usa `anyio.from_thread.run` para ejecutar código asíncrono desde una función sincrónica, ya que los endpoints definidos en la API no son `async`.

Subtarea 4.1.2: Frontend

3. [hooks/useWebSocket.js](#)

Este custom hook se encarga de abrir la conexión al WebSocket y escuchar mensajes entrantes:

```

src > hooks > JS useWebSocket.js > ...
1  import { useEffect } from "react";
2  import API_URL from "../api/config";
3
4  export default function useWebSocket(onMessageCallback) {
5    useEffect(() => {
6      // Detectar protocolo correcto
7      const wsProtocol = window.location.protocol === "https:" ? "wss" : "ws";
8      const socketUrl = `${wsProtocol}://${API_URL.replace(
9        /^https?:\/\//,
10        ""
11      )}/ws/movimientos`;
12
13      // Montaje del websocket
14      const socket = new WebSocket(socketUrl);
15
16      socket.onmessage = (event) => {
17        console.log("Mensaje recibido:", event.data);
18        onMessageCallback(event.data);
19      };
20
21      return () => {
22        socket.close();
23      };
24    }, [onMessageCallback]);
25  }

```

Figura 41: Captura del hook useWebSocket()

El protocolo wss o ws se selecciona automáticamente según si se está en HTTPS o HTTP.

4. components/Navbar.jsx

Aquí se usa el hook useWebSocket para escuchar notificaciones y actualizar el estado local de la campaña:

```

37  // Websocket
38  useWebSocket((mensaje) => {
39    setNotificaciones((prev) => {
40      const actualizadas = [mensaje, ...prev.slice(0, 4)];
41      localStorage.setItem("notificaciones", JSON.stringify(actualizadas));
42      return actualizadas;
43    });
44    setNotificacionActiva(true);
45  });

```

Figura 42: Detalle en Navbar.jsx de la llamada al hook useWebSocket()

Cuando el usuario hace clic en el icono de la campana, se muestra un dropdown con las notificaciones más recientes, y se resalta visualmente si hay una nueva no leída. Al hacer clic en cualquiera de ellas, se redirige al usuario a la página del listado de movimientos.

Subtarea 4.2: Refactorización del código y testeo

Durante esta fase se hace una revisión del código implementado en las tareas anteriores con el objetivo de mejorar su legibilidad, mantener la coherencia del estilo de desarrollo y garantizar su correcto funcionamiento. La refactorización incluyó la reorganización de algunos fragmentos de lógica, la eliminación de redundancias y la aplicación de buenas prácticas tanto en el backend como en el frontend.

Además, se corrigieron errores detectados durante las pruebas manuales iniciales, asegurando una integración fluida del WebSocket con el resto del sistema. Se verificó especialmente que las notificaciones en tiempo real funcionaran correctamente en distintos escenarios.

Estas pruebas permitieron confirmar la estabilidad de la funcionalidad implementada, así como su compatibilidad con otras partes críticas del sistema, como el manejo de sesiones y el almacenamiento local del historial de notificaciones. También se validó que el WebSocket no generara errores silenciosos ni dejara conexiones abiertas al cerrar la aplicación.

Subtarea 4.3: Creación de repositorios públicos en GitHub

Como parte del proceso de publicación y despliegue de la aplicación, se decidió crear una estructura organizada de repositorios públicos en GitHub, separando el backend y el frontend en proyectos independientes y centralizando el acceso desde un repositorio principal.

Hasta ese momento, el desarrollo se había realizado en repositorios privados. Por ello, se optó por crear nuevos repositorios "limpios", sin el historial de commits previo, con el objetivo de ofrecer una presentación profesional y clara del proyecto, tanto para su despliegue como para su documentación pública.

Los tres repositorios creados fueron:

- [tabula \(repositorio principal\)](#): contiene únicamente un README.md bilingüe (español e inglés), donde se presentan los objetivos del proyecto, las tecnologías utilizadas, sus funcionalidades principales y enlaces directos a los repositorios del frontend y backend. Este repositorio actúa como punto de entrada para quienes deseen conocer o clonar la aplicación.
- [tabula-backend](#): incluye todo el código del servidor, desarrollado con FastAPI. El README explica cómo configurar el entorno virtual, variables de entorno, conexión con PostgreSQL, y cómo ejecutar la API localmente.
- [tabula-frontend](#): contiene el código del cliente desarrollado en React. El README guía al usuario en la instalación de dependencias y configuración del entorno (.env).

En los tres repositorios se añadieron las licencias correspondientes (GNU GPL v3.0) y documentación detallada para facilitar tanto su instalación local como su revisión técnica por terceros.

Desde estos repositorios públicos se realizó el despliegue final de la aplicación, tal y como se detalla en la siguiente subtarea.

Subtarea 4.4: Configuración del hosting en Render y Vercel

Inicialmente se planteó desplegar tanto el backend como el frontend en un mismo servidor utilizando contenedores Docker. Esta opción permitiría tener control completo sobre la infraestructura y una integración más cercana entre ambos servicios. Sin embargo, esta alternativa implicaba una curva de aprendizaje adicional, ya que Docker es una tecnología que aún no se domina del todo, y no se disponía del tiempo necesario para una formación adecuada durante esta fase final del proyecto.

Por ello, se optó por una solución más práctica y rápida: **separar el frontend y el backend en dos servidores distintos**, usando plataformas especializadas para cada uno. El frontend se desplegó en **Vercel** y el backend en **Render**, funcionando este último como un microservicio independiente que expone una API REST consumida por el cliente React.

Subtarea 4.4.1: Backend – Despliegue en Render

Para alojar el servidor de FastAPI, se utilizó la plataforma **Render.com**, que permite desplegar fácilmente aplicaciones en Python con integración directa desde GitHub. El proceso fue sencillo y consistió en los siguientes pasos:

1. **Importación del repositorio:** Se conectó la cuenta de GitHub con Render, importando el repositorio público [tabula-backend](#).
2. **Creación del servicio web:**
 - Se seleccionó la opción *Web Service*.
 - Render detectó automáticamente Python como entorno de ejecución.
 - Se configuró el comando de arranque de FastAPI con Uvicorn:

```
uvicorn app.main:app --host 0.0.0.0 --port 10000
```

3. **Base de datos PostgreSQL:** Se creó una base de datos en Render, y se enlazó con la aplicación mediante la variable de entorno `DATABASE_URL`. El trigger se configuró mediante PgAdmin conectado a la base de datos remota, ya que Render no permite realizar este tipo de acciones directamente sobre la base de datos desde su interfaz.

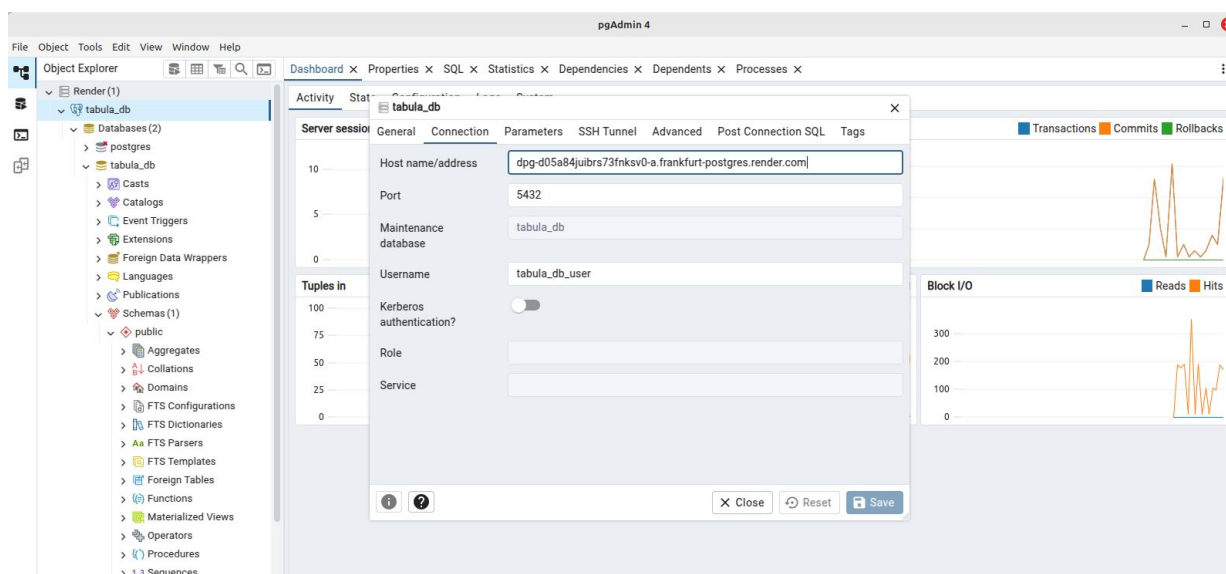


Figura 43: Captura desde PgAdmin, se ve la conexión a `tabula_db` desplegada en Render

4. **Variables de entorno:** Se añadieron otras variables necesarias para el correcto funcionamiento del backend, como `SECRET_KEY`, `ALGORITHM`, o la configuración del CORS.

5. **Deploy automático:** Render realiza el despliegue automáticamente cada vez que se detecta un nuevo commit en la rama principal del repositorio.

Render también ofrece soporte nativo para WebSocket, por lo que no fue necesario realizar configuraciones adicionales para habilitar las notificaciones en tiempo real desde el backend.

SERVICE NAME	2	STATUS	RUNTIME
🌐	<code>tabula-backend</code>	✓ Deployed	Python 3
📄	<code>tabula_db</code>	✓ Available	PostgreSQL 16

Figura 44: Captura desde Render.com, se pueden ver los dos servicios desplegados: servidor `tabula-backend` y base de datos `tabula_db`

Subtarea 4.4.2: Frontend – Despliegue en Vercel

El frontend de la aplicación se despliega en **Vercel**, una plataforma optimizada para aplicaciones frontend modernas y que ofrece integración directa con GitHub. Al igual que en el caso del backend, el proceso de despliegue fue fácil e intuitivo:

1. **Importación del repositorio:** Se conectó la cuenta de GitHub a Vercel, seleccionando el repositorio público [tabula-frontend](#).
2. **Detección automática del entorno:** Vercel detectó automáticamente que el proyecto estaba construido con Vite, por lo que configuró los scripts de build y los ajustes necesarios sin intervención manual.
3. **Variables de entorno:** Se añadieron variables como **VITE_API_URL**, apuntando a la URL del backend desplegado en Render. Esto permitió que el frontend consumiera correctamente la API sin necesidad de cambios adicionales en el código fuente.
4. **Corrección de rutas (SPA):** Para que las rutas internas de React funcionaran correctamente incluso al hacer refresh o al acceder directamente a una subruta (por ejemplo, /productos/listado), fue necesario crear un archivo adicional llamado [vercel.json](#) con el siguiente contenido:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Este archivo asegura que todas las rutas del navegador se redirijan al archivo principal de la aplicación (index.html), tal como requiere cualquier aplicación de una sola página (SPA).

5. **Deploy automático:** A partir de este punto, cada cambio enviado al repositorio principal de GitHub genera automáticamente un nuevo despliegue en Vercel.

El resultado fue un frontend accesible desde cualquier navegador, completamente integrado con el backend alojado en Render, y con soporte para autenticación, rutas protegidas, y notificaciones en tiempo real mediante WebSockets.

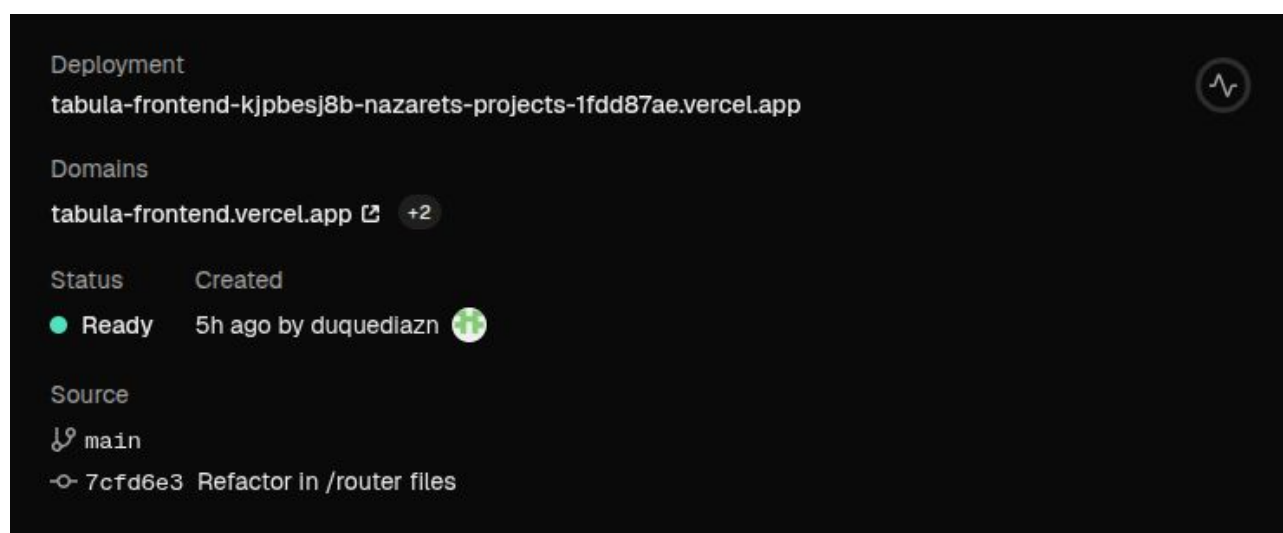


Figura 45: Captura detalle del frontend desplegado en Vercel

Duración

Tarea/Subtarea	Duración (horas)	Fecha inicio	Fecha final
Tarea 4: WebSocket y Despliegue		21/04/2025	27/04/2025
Subtarea 4.1: Desarrollo de la funcionalidad de WebSocket		21/04/2025	23/04/2025
4.1.1: Backend	2,5	21/04/2025	22/04/2025
4.1.2: Frontend	1,5	22/04/2025	23/04/2025
Subtarea 4.2: Refactorización del código y testeo	4,5	23/04/2025	27/04/2025
Subtarea 4.3: Creación de repositorios públicos en GitHub	1	27/04/2025	27/04/2025
Subtarea 4.4: Configuración del hosting en Render y Vercel		27/04/2025	27/04/2025
4.4.1: Backend – Despliegue en Render	0,25	27/04/2025	27/04/2025
4.4.2: Frontend – Despliegue en Vercel	0,25	27/04/2025	27/04/2025
TOTAL	10		

RECURSOS HUMANOS

Aunque el desarrollo de este proyecto ha sido llevado a cabo por una única persona, en un entorno profesional un sistema de estas características requeriría la participación de varios perfiles especializados para asegurar una implementación eficiente, escalable y mantenible. A continuación se detallan los roles típicos necesarios:

Perfil Profesional	Función Principal	Nº de personas
Desarrollador/a Backend	Diseño de la API, base de datos, autenticación, lógica de negocio y WebSocket	1
Desarrollador/a Frontend	Diseño de interfaz de usuario, integración con API, diseño responsive, gráficos y notificaciones	1
Diseñador/a UX/UI	Diseño visual, usabilidad, accesibilidad, estilo y experiencia de usuario	0–1 (no incluido en este proyecto)
DevOps / Infraestructura	Contenedorización con Docker, despliegue, seguridad, monitorización	0–1 (no incluido en este proyecto)
Tester / QA	Validación funcional, pruebas automatizadas y control de calidad	0–1 (no incluido en este proyecto)

En este caso, todos los roles fueron asumidos por la misma persona, lo que ha implicado un mayor esfuerzo individual pero ha permitido adquirir una visión integral del desarrollo de software full-stack.

RECURSOS MATERIALES

Para el desarrollo y despliegue de este proyecto se han utilizado los siguientes recursos materiales:

- **Equipo informático personal** con conexión a Internet.
- **Entorno de desarrollo local:**
 - Visual Studio Code (editor de código)
 - Python 3.11
 - Node.js + npm
 - PostgreSQL (local o alojado)
- **Servicios y plataformas en la nube:**
 - GitHub (control de versiones, repositorios públicos)
 - Render (hosting gratuito del backend + base de datos PostgreSQL)
 - Vercel (hosting gratuito del frontend en React)
- **Herramientas de testing y depuración:**
 - Thunder Client integrado en VS Code
 - Lighthouse de Google Chrome para pruebas de accesibilidad y rendimiento
- **Otras herramientas:**
 - Docker (planificado para futuro despliegue)

CRONOGRAMA

La siguiente gráfica de Gantt representa la planificación temporal del proyecto, estructurada en cinco tareas principales desarrolladas a lo largo de doce semanas. Cada fase incluye subtareas específicas e hitos que marcan la finalización de bloques funcionales clave.

La ejecución del proyecto se llevó a cabo según lo planificado, cumpliéndose todos los hitos dentro de los plazos establecidos. La documentación se fue elaborando en paralelo al desarrollo técnico, aunque se intensificó en la fase final, junto con la preparación de la presentación y material audiovisual.

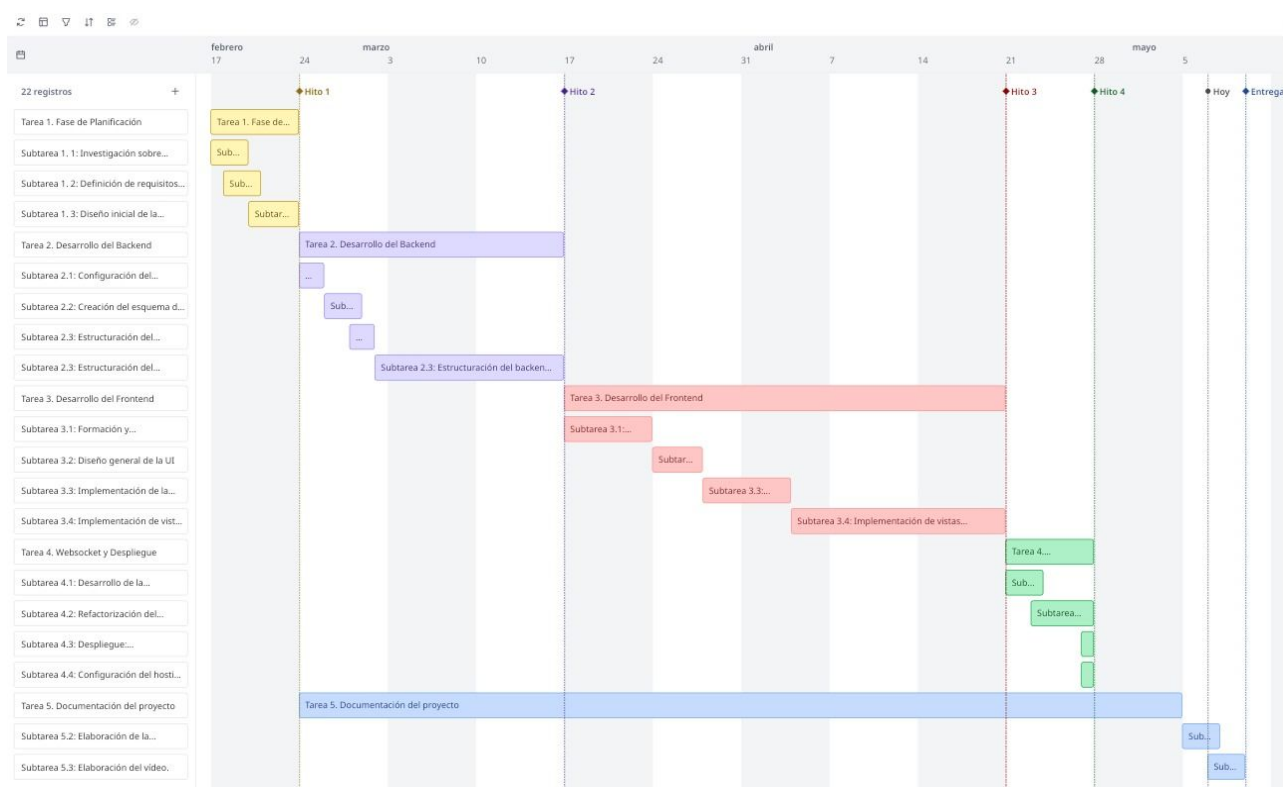


Figura 46: Gráfico de Gantt del proyecto, realizado en miro.com

PRESUPUESTO

Aunque el desarrollo de este proyecto ha sido realizado de forma individual y utilizando recursos gratuitos, a efectos de estimación profesional es necesario contemplar los costes que implicaría su ejecución en un entorno empresarial real. A continuación, se detalla un presupuesto aproximado por tarea y subtareas, incluyendo tanto recursos humanos como materiales.

Tarea	Recursos Humanos (€)	Recursos Materiales (€)	Total (€)
Tarea 1: Análisis inicial y diseño de la arquitectura del sistema	150 € (6 h)	0 €	150 €
Subtareas: Investigación, definición de requisitos, diseño arquitectónico			
Tarea 2: Desarrollo del Backend (Base de Datos y API REST)	1500 € (60 h)	50 € (hosting backend + BBDD)	1550 €
Subtareas: Configuración, diseño BBDD, API REST con FastAPI			
Tarea 3: Desarrollo del Frontend (React y Tailwind)	2250 € (90 h)	10 € (hosting frontend + herramientas)	2260 €
Subtareas: Formación, diseño UI, autenticación, vistas y componentes			
Tarea 4: WebSocket y Despliegue	250 € (10 h)	25 € (dominio + ajustes despliegue)	275 €
Subtareas: Notificaciones en tiempo real, despliegue en Render y Vercel			
TOTAL PROYECTO	4.150,00 €	85,00 €	4.235,00 €

Consideraciones:

- **Tarifa estimada:** 25€/h. En un entorno real puede variar entre 20€/h y 60€/h dependiendo del nivel de especialización.
- **Infraestructura:**
 - Hosting backend (Render, DigitalOcean...): estimado en ~5–10€/mes

- Hosting frontend (Vercel, Netlify...): gratuito o ~5–10€/mes en planes de pago
- Dominio propio: ~10–15€/año
- **Licencias:** No se contemplan gastos, ya que todo el stack utilizado es de código abierto.
- **Mantenimiento futuro:** Podría añadirse un contrato mensual (~100€/mes) si se tratara de un entorno de producción.

POLÍTICA DE SEGUIMIENTO Y EVALUACIÓN

Durante todo el desarrollo del proyecto se aplicaron medidas de seguimiento que permitieron detectar desviaciones y tomar decisiones a tiempo. Las principales acciones realizadas fueron:

- **Planificación semanal:** Cada semana se definieron tareas concretas, con fechas límite y objetivos claros.
- **Control del avance mediante tareas:** Gracias a la elaboración del gráfico Gantt del apartado anterior, se pudo llevar un seguimiento detallado en el progreso de las tareas y control de fechas límite.
- **Autoevaluación continua:** Al trabajar de forma individual, se estableció una rutina de revisión del trabajo finalizado al final de cada jornada para verificar que se cumplían los requisitos establecidos.
- **Versionado con Git:** El uso de Git permitió mantener el control sobre los cambios realizados, revertir errores y organizar el código en versiones funcionales.
- **Pruebas manuales frecuentes:** Cada funcionalidad fue probada inmediatamente después de su desarrollo, especialmente en flujos críticos como autenticación, refresco de token, gestión de movimientos o stock.
- **Documentación en paralelo:** Se documentó el proceso conforme avanzaba el desarrollo, facilitando así la validación de objetivos cumplidos y la identificación de mejoras pendientes.

Futuras mejoras

Aunque el proyecto cumple con los objetivos planteados, se han identificado varias líneas de mejora que podrían implementarse en futuras versiones:

- **Seguridad del WebSocket:** Actualmente el canal WebSocket está abierto. Se recomienda implementar autenticación y control de acceso para restringir el envío y recepción de mensajes a usuarios autorizados.

- **Refactorización del frontend:** Mejorar la organización del código en React, separando lógica de presentación, utilizando más hooks reutilizables y modularizando componentes.
- **Mejora de la accesibilidad y diseño:** Aunque se han aplicado buenas prácticas, siempre es posible mejorar aspectos como contraste, navegación por teclado o etiquetas semánticas. Se recomienda realizar auditorías periódicas con herramientas como Lighthouse.
- **Optimización del rendimiento:** Revisión del uso de recursos, lazy loading de rutas y componentes, y análisis del tiempo de carga general.
- **Despliegue con Docker:** Preparar contenedores Docker para frontend y backend, lo que facilitaría el despliegue en servidores personalizados o entornos CI/CD.
- **Más funcionalidades en tiempo real:** Ampliar el uso de WebSocket para incluir notificaciones por usuario, sincronización entre sesiones activas o indicadores de actividad, o actualización en tiempo real de las gráficas.
- **Autenticación más robusta:** Implementar expiración forzada de tokens, detección de múltiples sesiones, y mecanismos de recuperación de cuenta mediante correo electrónico.
- **Test automatizados:** Añadir pruebas unitarias y de integración, tanto en backend como en frontend, para mejorar la calidad del código y prevenir regresiones.
- **Internacionalización (i18n):** Preparar el frontend para soportar múltiples idiomas, mejorando la accesibilidad global.

ANEXOS

Junto con el presente documento se anexan también los recursos pedidos:

- **Presentación** para la defensa del proyecto, en formato **LibreOffice Impress**
- **VideoTutorial**, con explicación del proyecto y demostración de la aplicación desarrollada.
- **Fichero comprimido (tabula.zip)** con el código fuente completo de la aplicación. Cada parte del proyecto contiene un **README.md** con explicaciones detalladas para su puesta en funcionamiento si se desea probar el local.
- Además, se adjunta el enlace a los repositorios públicos del proyecto en GitHub:
 - Repositorio principal: <https://github.com/duquediazn/tabula>
 - Repositorio del backend: <https://github.com/duquediazn/tabula-backend>
 - Repositorio del frontend: <https://github.com/duquediazn/tabula-frontend>
- Como el proyecto final se encuentra desplegado, se puede visitar en:
<https://tabula-frontend.vercel.app/login>

Requiere autenticación, pueden usarse las siguientes credenciales:

- usuario: admin@example.com
- password: mi_contraseña_segura

BIBLIOGRAFÍA

Recursos y referencias textuales externas

- [1] - Cornago, I. S. "Arqueología de la escritura: tablillas enceradas en el mundo romano." *Palaeohispánica* 21 (2021):
<https://ifc.dpz.es/recursos/publicaciones/39/22/08cornago.pdf#:~:text=match%20at%20L1523%20contabilidad%20,y%20en%20la%20ense%C3%B1anza%20C115%20algunos>
- [2] - Bigger Applications – Multiple Files: <https://fastapi.tiangolo.com/tutorial/bigger-applications/>
- [3] – How to Structure Your FastAPI Projects:
<https://medium.com/@amirm.lavasani/how-to-structure-your-fastapi-projects-0219a6600a8f>
- [4] – Virtual Environments: <https://fastapi.tiangolo.com/virtual-environments/>
- [5] – How to Create Requirements.txt File in Python:
<https://www.geeksforgeeks.org/how-to-create-requirements-txt-file-in-python/>
- [6] – SQL Relational Databases: <https://fastapi.tiangolo.com/tutorial/sql-databases/>
- [7] - OAuth2 with Password (and hashing), Bearer with JWT tokens:
<https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>
- [8] – Introduction to JSON Web Tokens: <https://jwt.io/introduction>
- [9] – Passing Data Deeply with Context: <https://react.dev/learn/passing-data-deeply-with-context>

Documentación oficial y guías de las tecnologías y herramientas del proyecto

- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://es.react.dev/learn>
- Vite: <https://vite.dev/guide/>
- Tailwind CSS: <https://tailwindcss.com/docs/installation/using-vite>
- PostgreSQL: <https://www.postgresql.org/docs/>
- PgAdmin: <https://www.pgadmin.org/docs/>
- Recharts: <https://recharts.org/en-US/guide>
- VS Code: <https://code.visualstudio.com/docs>
- Thunder Client: <https://docs.thunderclient.com/>
- Git: <https://git-scm.com/doc>
- Miro.com (aplicación web para la creación de gráficos): <https://miro.com/>

Imágenes y recursos visuales (licencias y atribuciones):

- Logo React, por ReactJS - <https://react.dev>, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=63486148>
- Logo Tailwind, por Tailwind Labs Inc. - File: <https://tailwindcss.com/brand/tailwindcss-logotype.zipPage>: <https://tailwindcss.com/brand>, Dominio público, <https://commons.wikimedia.org/w/index.php?curid=115514724>
- Logo FastApi, por Sebastián Ramírez - <https://github.com/fastapi/fastapi/blob/master/docs/en/docs/img/logo-margin/logo-teal.svg>, Dominio público, <https://commons.wikimedia.org/w/index.php?curid=152137590>
- Logo de PostgreSQL, por Daniel Lundin - https://wiki.postgresql.org/images/a/a4/PostgreSQL_logo.3colors.svg, PostgreSQL License, <https://commons.wikimedia.org/w/index.php?curid=1395316>